

ECOMMERCE RECOMMENDATION ENGINE

Báo cáo đồ án: PYTHON CHO KHOA HỌC DỮ LIỆU

Đề tài: *Ecommerce Recommendation Engine From Customer Metrics to Personalized Suggestions*

Lớp: 23TTH - Khoa Toán - Tin học

Giảng viên bộ môn: ThS. Hà Văn Thảo

Thành viên nhóm:

1. 22110193 - Nguyễn Thị Phương Tâm

2. 22110234 - Vương Mỹ Trang

[Link Dataset: Online Retail](#)

1. GIỚI THIỆU

Báo cáo này sử dụng tập dữ liệu giao dịch thương mại điện tử từ kho dữ liệu UCI Machine Learning Repository với tên "Online Retail". Dữ liệu bao gồm toàn bộ giao dịch từ 01/12/2010 đến 09/12/2011 của một doanh nghiệp bán lẻ trực tuyến có trụ sở tại Vương Quốc Anh, chuyên cung cấp các sản phẩm quà tặng độc đáo, với đối tượng khách hàng chủ yếu là các nhà bán buôn. Tập dữ liệu được cung cấp bởi Dr. Daqing Chen (London South Bank University). Đây là nguồn dữ liệu thực tế, có tính tổng hợp cao và phù hợp cho các bài toán phân tích chuỗi thời gian, phân cụm, phân loại và xây dựng hệ thống gợi ý trong bối cảnh thương mại điện tử.

2. KHAI BÁO DỮ LIỆU VÀ CÁC THƯ VIỆN CẦN THIẾT

```
In [1]: import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import plotly.graph_objects as go
from matplotlib.colors import LinearSegmentedColormap
from matplotlib import colors as mcolors
from scipy.stats import linregress
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from yellowbrick.cluster import KElbowVisualizer, SilhouetteVisualizer
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from sklearn.cluster import KMeans
from tabulate import tabulate
from collections import Counter

%matplotlib inline
```

3. TỔNG QUAN VỀ DỮ LIỆU

```
In [2]: df = pd.read_csv('data.csv', encoding='ISO-8859-1')
df.head(10)
```

Out[2]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	12/1/2010 8:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	12/1/2010 8:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	12/1/2010 8:26	3.39	17850.0	United Kingdom
5	536365	22752	SET 7 BABUSHKA NESTING BOXES	2	12/1/2010 8:26	7.65	17850.0	United Kingdom
6	536365	21730	GLASS STAR FROSTED T-LIGHT HOLDER	6	12/1/2010 8:26	4.25	17850.0	United Kingdom
7	536366	22633	HAND WARMER UNION JACK	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
8	536366	22632	HAND WARMER RED POLKA DOT	6	12/1/2010 8:28	1.85	17850.0	United Kingdom
9	536367	84879	ASSORTED COLOUR BIRD ORNAMENT	32	12/1/2010 8:34	1.69	13047.0	United Kingdom

Ta dùng lệnh `info()` để kiểm tra các thông số tổng quan về dữ liệu này

```
In [3]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 541909 entries, 0 to 541908
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   InvoiceNo        541909 non-null object
1   StockCode       541909 non-null object
2   Description      540455 non-null object
3   Quantity         541909 non-null int64
4   InvoiceDate      541909 non-null object
5   UnitPrice       541909 non-null float64
6   CustomerID      406829 non-null float64
7   Country         541909 non-null object
dtypes: float64(2), int64(1), object(5)
memory usage: 33.1+ MB
```

Dựa vào kết quả trên, ta có bộ dữ liệu có tổng cộng 541,909 dòng và 8 cột dữ liệu bao gồm: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country.

Ngoài ra, ta thấy cột CustomerID chỉ có 406,829 giá trị không trống. Điều này có nghĩa là có khoảng 135,080 giao dịch (chiếm ~25%) không có mã khách hàng và cột Description: Có 1,454 dòng bị thiếu mô tả sản phẩm.

Ý nghĩa cơ bản của từng cột:

1. InvoiceNo: Mã đại diện cho mỗi giao dịch duy nhất. Nếu mã này bắt đầu bằng chữ 'C', là ký hiệu của một đơn hàng bị hủy bỏ.
2. StockCode: Mã sản phẩm, được gán duy nhất cho từng loại mặt hàng riêng biệt.
3. Description: Tên hoặc mô tả chi tiết của từng loại sản phẩm.
4. Quantity: Số lượng đơn vị sản phẩm trong mỗi giao dịch.
5. InvoiceDate: Ngày và giờ chính xác khi giao dịch phát sinh.
6. UnitPrice: Đơn giá của sản phẩm tính bằng đồng bảng Anh (£).
7. CustomerID: Mã định danh duy nhất được gán cho mỗi khách hàng.
8. Country: Quốc gia nơi khách hàng cư trú.

Tạo thống kê tóm tắt để có cái nhìn ban đầu về phân phối dữ liệu:

```
In [4]: # Thống kê mô tả cho các biến số
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Quantity	541909.0	9.552250	218.081158	-80995.00	1.00	3.00	10.00	80995.0
UnitPrice	541909.0	4.611114	96.759853	-11062.06	1.25	2.08	4.13	38970.0
CustomerID	406829.0	15287.690570	1713.600303	12346.00	13953.00	15152.00	16791.00	18287.0

Nhìn vào bảng thống kê trên ta thấy:

Cột Quantity:

- Giá trị nhỏ nhất (min = -80,995): Điều này xác nhận giả thuyết rằng dữ liệu chứa các đơn hàng bị hủy hoặc trả hàng (thường bắt đầu bằng chữ 'C' trong InvoiceNo).
- Độ lệch chuẩn (std = 218) cao gấp nhiều lần giá trị trung bình (mean = 9.55). Điều này cho thấy sự biến động cực lớn: đa số khách mua ít (50% khách mua từ 3 sản phẩm trở xuống), nhưng có những đơn hàng lên tới 80,995 sản phẩm (max).

Cột UnitPrice:

- Giá trị nhỏ nhất (min = -11,062.06): Điều này cho thấy sự bất thường của dữ liệu hoặc lỗi hệ thống vì đơn giá không thể âm.
- Giá trị lớn nhất (max = 38,970): Điều này cho thấy có những mặt hàng có giá cao bất thường so với mức trung bình là 4.61.
- 75% sản phẩm có giá dưới 4.13 bảng Anh. Điều này cho thấy đây là cửa hàng bán lẻ các mặt hàng tiêu dùng nhỏ lẻ là chủ yếu.

Cột CustomerID:

- Chỉ có 406,829 dòng. Nghĩa là có khoảng 25% dữ liệu không biết khách hàng là ai.
- Dải ID: Mã khách hàng nằm trong khoảng từ 12,346 đến 18,287.

```
In [5]: # Thống kê mô tả các biến còn lại
df.describe(include='object').T
```

```
Out[5]:
```

	count	unique	top	freq
InvoiceNo	541909	25900	573585	1114
StockCode	541909	4070	85123A	2313
Description	540455	4223	WHITE HANGING HEART T-LIGHT HOLDER	2369
InvoiceDate	541909	23260	10/31/2011 14:41	1114
Country	541909	38	United Kingdom	495478

Bảng thống kê trên ta thấy:

Cột InvoiceNo:

- Có 25,900 hóa đơn duy nhất trên tổng số hơn 541,000 dòng dữ liệu. Điều này cho thấy khách hàng thường mua nhiều mặt hàng chứ không chỉ mua đơn lẻ một món.

Cột StockCode và Description:

- Có 4,070 mã sản phẩm (StockCode) nhưng lại có tới 4,223 mô tả (Description). Điều này cho thấy có hiện tượng một mã sản phẩm nhưng có nhiều cách viết mô tả khác nhau, hoặc có các mô tả rác.
- Sản phẩm "WHITE HANGING HEART T-LIGHT HOLDER" là mặt hàng phổ biến nhất, xuất hiện trong 2,369 giao dịch.

Cột InvoiceDate:

- Có 23,260 mốc thời gian khác nhau.
- Giá trị top (10/31/2011 14:41) xuất hiện 1,114 lần. Nghĩa là có 1,114 giao dịch xảy ra trong cùng 1 phút. Khả năng cao đây là một đợt import dữ liệu hàng loạt hoặc các đơn hàng sỉ được xử lý cùng một lúc.

Cột Country:

- Có 38 quốc gia khác nhau, nhưng United Kingdom xuất hiện tới 495,478 lần trên tổng số 541,909 dòng. Thị trường Anh chiếm khoảng 91.4% toàn bộ dữ liệu. Điều này cho thấy đây là một doanh nghiệp bán lẻ nội địa tại Anh là chủ yếu.

4. XỬ LÝ DỮ LIỆU

4.1 Xử lý giá trị bị thiếu:

Đầu tiên ta xác định tỷ lệ phần trăm giá trị bị thiếu trong từng cột:

Ta dùng `df.isnull().sum()` để kiểm tra từng ô trong bảng, nếu ô nào trống (null) thì trả về giá trị True, ngược lại là False.

Sau đó cộng dồn các giá trị True lại. Kết quả là tổng số dòng bị trống cho mỗi cột lưu vào biến `missing_data`.

```
In [6]: # Tính số lượng giá trị bị thiếu (NaN) cho từng cột trong DataFrame
missing_data = df.isnull().sum()

# Tính tỷ lệ phần trăm giá trị bị thiếu
# Chỉ giữ lại các cột có ít nhất 1 giá trị bị thiếu
missing_percentage = (missing_data[missing_data > 0] / df.shape[0]) * 100

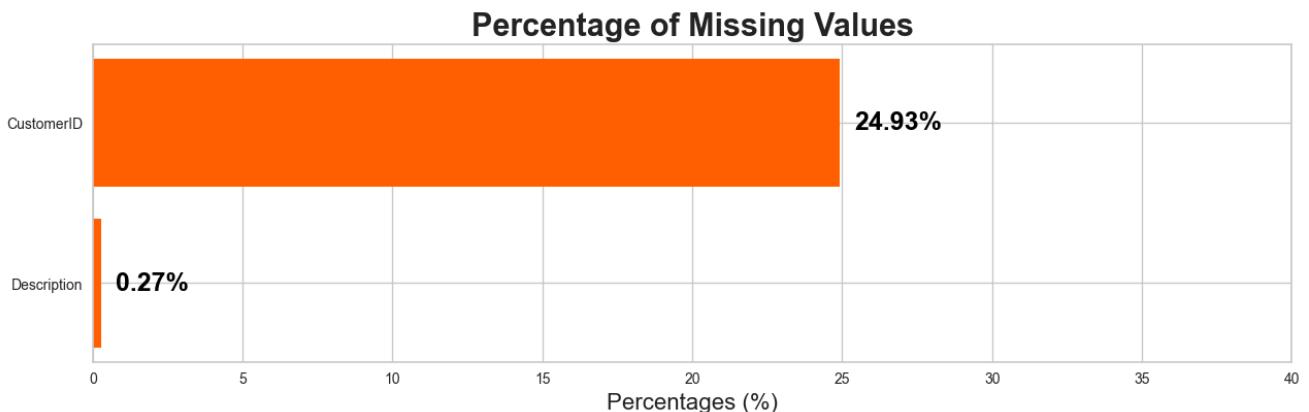
# Sắp xếp các cột theo tỷ lệ thiếu tăng dần (nhỏ → lớn)
```

```
missing_percentage.sort_values(ascending=True, inplace=True)

# Tạo biểu đồ thanh ngang (horizontal bar chart)
fig, ax = plt.subplots(figsize=(15, 4))
ax.barh(missing_percentage.index, missing_percentage, color='#ff6200')

for i, (value, name) in enumerate(zip(missing_percentage, missing_percentage.index)):
    ax.text(value+0.5, i, f"{value:.2f}%", ha='left', va='center', fontweight='bold', fontsize=18, color='black')

ax.set_xlim([0, 40])
plt.title("Percentage of Missing Values", fontweight='bold', fontsize=22)
plt.xlabel('Percentages (%)', fontsize=16)
plt.show()
```



Đây là biểu đồ thanh ngang dùng để trực quan hóa kết quả từ việc kiểm tra dữ liệu bị thiếu. ta thấy:

CustomerID (24,93% giá trị bị thiếu):

- Cột CustomerID chứa gần một phần tư dữ liệu bị thiếu. Cột này rất quan trọng để phân nhóm khách hàng và tạo hệ thống gợi ý. Việc điền vào một tỷ lệ lớn giá trị bị thiếu như vậy có thể dẫn đến sai lệch hoặc nhiễu đáng kể trong phân tích.
- Vì mục tiêu của bài toán là phân cụm khách hàng, việc thiếu CustomerID khiến dữ liệu không thể định danh. Do đó, chúng ta loại bỏ 24.93% dòng thiếu này để đảm bảo mô hình phân cụm hoạt động chính xác nhất.

Cột Description (0,27% giá trị bị thiếu):

- Cột Description có một tỷ lệ nhỏ giá trị bị thiếu. Tuy nhiên, đã nhận thấy rằng có những bất nhất giữa StockCode và Description trong dữ liệu, việc tự động lấp đầy dữ liệu sẽ làm giảm độ tin cậy của tập dữ liệu.
- Hơn nữa, vì tỷ lệ bị thiếu khá thấp, việc loại bỏ các hàng này là giải pháp tối ưu để bảo vệ tính chính xác cho các phân tích về sau.

Phương án xử lý dữ liệu bị thiếu:

Do đó, phương án cuối cùng là loại bỏ các dòng có dữ liệu bị thiếu ở cột CustomerID hoặc Description.

Ta xem vài dòng dữ liệu có giá trị bị thiếu (NaN) ở CustomerID hoặc Description.

```
In [7]: df[df['CustomerID'].isnull() | df['Description'].isnull()].head()
```

Out[7]:	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country	
	622	536414	22139	NaN	56	12/1/2010 11:52	0.00	NaN	United Kingdom
	1443	536544	21773	DECORATIVE ROSE BATHROOM BOTTLE	1	12/1/2010 14:32	2.51	NaN	United Kingdom
	1444	536544	21774	DECORATIVE CATS BATHROOM BOTTLE	2	12/1/2010 14:32	2.51	NaN	United Kingdom
	1445	536544	21786	POLKADOT RAIN HAT	4	12/1/2010 14:32	0.85	NaN	United Kingdom
	1446	536544	21787	RAIN PONCHO RETROSPOT	2	12/1/2010 14:32	1.66	NaN	United Kingdom

Sau đó ta thực hiện loại bỏ các dòng có giá trị trống ở cột ở cột CustomerID hoặc cột Description.

```
In [8]: # Loại bỏ toàn bộ các dòng dữ liệu có giá trị bị thiếu (NaN)
df = df.dropna(subset=['CustomerID', 'Description'])

# Kiểm tra tổng số giá trị bị thiếu (NaN) còn lại trong toàn bộ DataFrame
df.isnull().sum().sum()
```

Out[8]: np.int64(0)

Kết quả cho ta thấy dữ liệu đã được xử lý không còn giá trị bị thiếu.

4.2 Xử lý giá trị trùng lặp:

Đầu tiên, ta dùng lệnh `df.duplicated(keep=False)` để xem có giá trị nào bị trùng trong bộ dữ liệu hay không.

Với `keep=False`, nó sẽ giữ lại tất cả các dòng tham gia vào việc trùng lặp (cả dòng gốc và các dòng bản sao)

```
In [9]: # Xác định các dòng dữ liệu bị trùng lặp trong DataFrame
duplicate_rows = df[df.duplicated(keep=False)]

# Sắp xếp các dòng trùng lặp theo các cột giúp dễ quan sát các dòng dữ liệu giống nhau nằm cạnh nhau
duplicate_rows_sorted = duplicate_rows.sort_values(by=['InvoiceNo', 'StockCode', 'Description', 'CustomerID'])

# Hiển thị 10 dòng đầu tiên để kiểm tra các dòng dữ liệu trùng lặp
duplicate_rows_sorted.head(10)
```

Out[9]:

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
494	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	12/1/2010 11:45	1.25	17908.0	United Kingdom
517	536409	21866	UNION JACK FLAG LUGGAGE TAG	1	12/1/2010 11:45	1.25	17908.0	United Kingdom
485	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	12/1/2010 11:45	4.95	17908.0	United Kingdom
539	536409	22111	SCOTTIE DOG HOT WATER BOTTLE	1	12/1/2010 11:45	4.95	17908.0	United Kingdom
489	536409	22866	HAND WARMER SCOTTY DOG DESIGN	1	12/1/2010 11:45	2.10	17908.0	United Kingdom
527	536409	22866	HAND WARMER SCOTTY DOG DESIGN	1	12/1/2010 11:45	2.10	17908.0	United Kingdom
521	536409	22900	SET 2 TEA TOWELS I LOVE LONDON	1	12/1/2010 11:45	2.95	17908.0	United Kingdom
537	536409	22900	SET 2 TEA TOWELS I LOVE LONDON	1	12/1/2010 11:45	2.95	17908.0	United Kingdom
578	536412	21448	12 DAISY PEGS IN WOOD BOX	1	12/1/2010 11:49	1.65	17920.0	United Kingdom
598	536412	21448	12 DAISY PEGS IN WOOD BOX	1	12/1/2010 11:49	1.65	17920.0	United Kingdom

Nhìn vào bảng trên ta thấy dòng số 494 và 517 đều có:

- Cùng mã đơn (InvoiceNo: 536409)
- Cùng mã hàng (StockCode: 21866)
- Cùng thời điểm mua (InvoiceDate: 12/1/2010 11:45)
- Cùng một khách hàng (CustomerID: 17908.0)
- Cùng số lượng và đơn giá.

Điều này cho thấy đây không phải là hai lần mua hàng độc lập, mà là một dòng dữ liệu bị trùng lặp trong hệ thống, nhiều khả năng phát sinh do lỗi lưu trữ hoặc quá trình nhập liệu.

Phương án:

Do đó, phương án xử lý được lựa chọn là chỉ giữ lại một dòng duy nhất cho mỗi nhóm dữ liệu trùng lặp và loại bỏ các dòng còn lại.

```
In [10]: # Loại bỏ các dòng dữ liệu trùng Lặp trong DataFrame
# inplace=True: thay đổi trực tiếp trên DataFrame df, không tạo bản sao mới
df.drop_duplicates(inplace=True)

# Trả về số Lượng dòng dữ liệu còn Lại sau khi đã Loại bỏ trùng Lặp
df.shape[0]
```

Out[10]: 401604

Ta thấy sau khi loại bỏ các dòng trùng lặp thì số dòng của dataFame còn 401604 dòng.

4.3 Xử lý các giao dịch bị hủy:

Ở bước đọc dữ liệu cho ta biết nếu mã mã đơn hàng bắt đầu bằng chữ 'C', là ký hiệu của một đơn hàng bị hủy bỏ.

Do đó ta dùng `np.where()` để phân biệt rõ ràng giữa đơn hàng Thành công và Bị hủy.

```
In [11]: # Tạo cột Transaction_Status để phân loại trạng thái giao dịch
# Nếu mã đơn hàng (InvoiceNo) bắt đầu bằng ký tự 'C' → giao dịch bị hủy (Cancelled)
# Ngược Lại → giao dịch hoàn tất (Completed)
df['Transaction_Status'] = np.where(df['InvoiceNo'].astype(str).str.startswith('C'), 'Cancelled', 'Completed')

# Lọc ra các dòng dữ liệu tương ứng với giao dịch bị hủy
cancelled_transactions = df[df['Transaction_Status'] == 'Cancelled']

# Thống kê mô tả các biến số của các giao dịch bị hủy
cancelled_transactions.describe().drop('CustomerID', axis=1)
```

```
Out[11]:
```

	Quantity	UnitPrice
count	8872.000000	8872.000000
mean	-30.774910	18.899512
std	1172.249902	445.190864
min	-80995.000000	0.010000
25%	-6.000000	1.450000
50%	-2.000000	2.950000
75%	-1.000000	4.950000
max	-1.000000	38970.000000

Từ bảng thống kê từ dữ liệu các giao dịch bị hủy trên ta thấy:

- Cột Quantity: tất cả các số lượng trong các giao dịch bị hủy đều là số âm, cho thấy đây thực sự là các đơn hàng đã bị hủy.
- Cột UnitPrice: Giá trị nhỏ nhất (min) là 0.01, cho thấy các đơn hàng bị hủy vẫn gắn liền với một giá trị sản phẩm cụ thể, không phải là các dòng điều chỉnh giá bằng 0.

Phương án xử lý các giao dịch bị hủy:

- Xét đến mục tiêu của dự án là phân nhóm khách hàng dựa trên hành vi và sở thích mua sắm của họ và cuối cùng tạo ra một hệ thống gợi ý, việc hiểu được các mô hình hủy giao dịch của khách hàng là rất quan trọng.
- Do đó, chiến lược là giữ các giao dịch bị hủy này trong bộ dữ liệu, cho phép hệ thống gợi ý có khả năng ngăn việc đề xuất các sản phẩm có khả năng cao bị hủy, từ đó cải thiện chất lượng gợi ý.

Tiếp theo ta tính tỷ lệ phần trăm các đơn hàng bị hủy:

```
In [12]: # Tính tỷ lệ (%) giao dịch bị hủy so với tổng số dòng dữ liệu
cancelled_percentage = (cancelled_transactions.shape[0] / df.shape[0]) * 100

# In ra kết quả
print(f"Tỷ lệ giao dịch bị hủy trong tập dữ liệu là: {cancelled_percentage:.2f}%")
```

Tỷ lệ giao dịch bị hủy trong tập dữ liệu là: 2.21%

4.4 Xử lý các điểm bất thường của mã sản phẩm (StockCode):

Để hiểu rõ hơn về mã sản phẩm, ta cần kiểm tra số lượng, định dạng, cấu trúc và mức độ phân bố của mã sản phẩm trong tập dữ liệu.

```
In [13]: # Đếm số lượng mã sản phẩm có giá trị duy nhất trong tập dữ liệu
unique_stock_codes = df['StockCode'].nunique()

# In ra kết quả
print(f"Số lượng mã cổ phiếu duy nhất trong tập dữ liệu là: {unique_stock_codes}")
```

Số lượng mã cổ phiếu duy nhất trong tập dữ liệu là: 3684

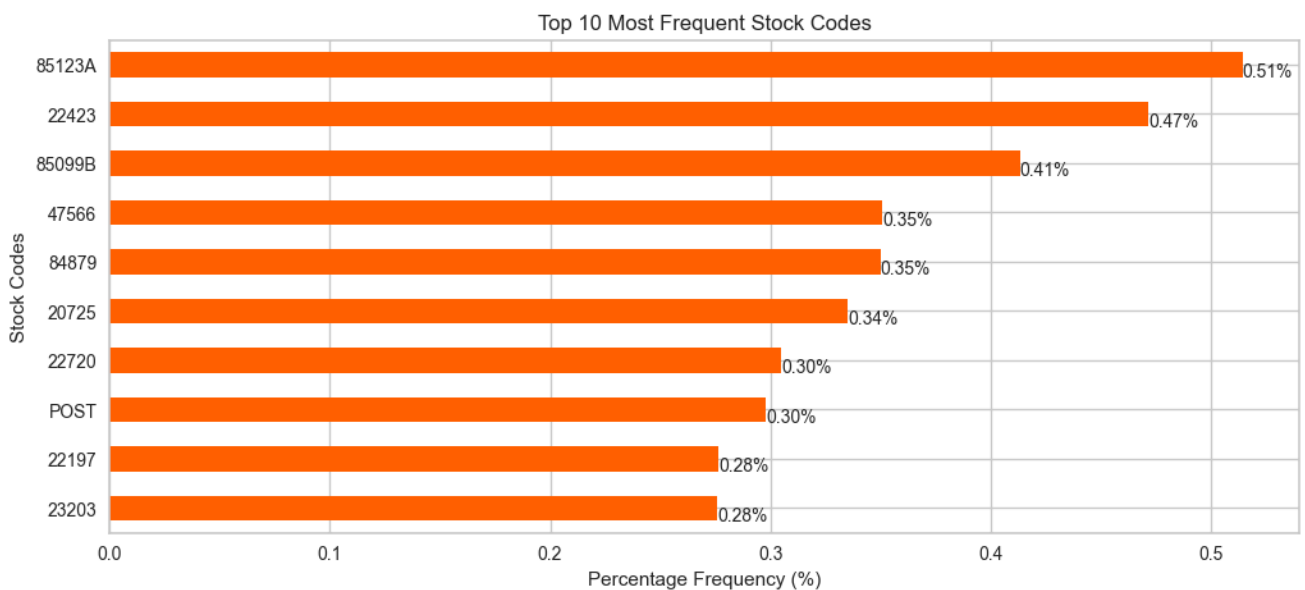
Trong 3684 mã đó, 10 mã nào xuất hiện nhiều nhất và chúng chiếm bao nhiêu % tổng lượng giao dịch?

```
In [14]: # Tính tần suất xuất hiện (%) của từng mã sản phẩm (StockCode)
# value_counts(normalize=True): trả về tỷ lệ xuất hiện (dạng phần trăm thập phân)
# head(10): Lấy 10 mã sản phẩm xuất hiện nhiều nhất
top_10_stock_codes = df['StockCode'].value_counts(normalize=True).head(10) * 100

# Vẽ biểu đồ cột ngang cho top 10 mã sản phẩm phổ biến nhất
plt.figure(figsize=(12, 5))
top_10_stock_codes.plot(kind='barh', color='#ff6200')

for index, value in enumerate(top_10_stock_codes):
    plt.text(value, index+0.25, f'{value:.2f}%', fontsize=10)

plt.title('Top 10 Most Frequent Stock Codes')
plt.xlabel('Percentage Frequency (%)')
plt.ylabel('Stock Codes')
plt.gca().invert_yaxis()
plt.show()
```



Nhìn vào biểu đồ ngang thể hiện top 10 mã sản phẩm xuất hiện nhiều nhất:

- Cho ta có thể cung cấp cái nhìn về các sản phẩm hoặc danh mục phổ biến mà khách hàng thường xuyên mua.
- Một số bất thường về Mã Hàng Hóa:
 - Có một số bất thường như mã 'POST'. Những bất thường này có thể đại diện cho dịch vụ hoặc giao dịch không phải sản phẩm (có thể là phí bưu chính) thay vì sản phẩm thực sự.
 - Các mã sản phẩm kèm thêm các ký tự 'A', 'B',...

Để đi sâu hơn vào việc xác định những bất thường này, ta tìm hiểu tần suất số lượng ký tự số trong mã sản phẩm, điều này có thể cung cấp cái nhìn về bản chất của những mục nhập bất thường này:

```
In [15]: # Lấy danh sách các mã sản phẩm (StockCode) duy nhất trong tập dữ liệu
unique_stock_codes = df['StockCode'].unique()

# Với mỗi mã sản phẩm, đếm số lượng ký tự dạng số (0-9) trong mã
# c.isdigit(): kiểm tra ký tự có phải là chữ số hay không
# value_counts(): thống kê tần suất của số lượng ký tự số
numeric_char_counts_in_unique_codes = pd.Series(unique_stock_codes).apply(lambda x: sum(c.isdigit()

# In ra bảng tần suất
print("Bảng tần suất số ký tự dạng số trong các mã cổ phiếu duy nhất:")
print(numeric_char_counts_in_unique_codes)
```

Bảng tần suất số ký tự dạng số trong các mã cổ phiếu duy nhất:

```
5    3676
0         7
1         1
```

Name: count, dtype: int64

Nhận xét:

- Ta thấy có một vài mã bất thường: có 7 mã cổ phiếu không chứa ký tự số nào và 1 mã cổ phiếu chỉ chứa 1 ký tự số. Những mã này sai lệch so với định dạng chuẩn và liệu chúng có đại diện cho các giao dịch sản phẩm hợp lệ hay không.
- Vì mục tiêu dự án là phân cụm dựa trên sản phẩm, nên ta cần xem danh tính 8 mã này là gì để có bước xử lý thích hợp.

```
In [16]: # Xác định các mã sản phẩm có cấu trúc bất thường có số ký tự dạng số trong mã bằng 0 hoặc 1
anomalous_stock_codes = [code for code in unique_stock_codes if sum(c.isdigit() for c in str(code))

# In ra danh sách
print("Các mã bất thường:")
for code in anomalous_stock_codes:
    print(code)
```

Các mã bất thường:

```
POST
D
C2
M
BANK CHARGES
PADS
DOT
CRUK
```

Các mã sản phẩm bất thường này chiếm bao nhiêu phần trăm trong toàn bộ dữ liệu?

```
In [17]: # Tính tỷ lệ (%) các dòng dữ liệu chứa mã sản phẩm bất thường
# df['StockCode'].isin(anomalous_stock_codes): kiểm tra mỗi dòng có thuộc nhóm mã bất thường hay không
percentage_anomalous = (df['StockCode'].isin(anomalous_stock_codes).sum() / len(df)) * 100

# In ra kết quả
print(f"Phần trăm các mã bất thường có trong dữ liệu: {percentage_anomalous:.2f}%")
```

Phần trăm các mã bất thường có trong dữ liệu: 0.48%

Chiến lược xử lý các mã sản phẩm bất thường

- Tỷ lệ 0.48% cho thấy các mã hàng bất thường chỉ chiếm một phần rất nhỏ trong tổng số giao dịch.
- Các mã này chủ yếu đại diện cho những giao dịch không phải sản phẩm thực tế (ví dụ: phí ngân hàng, phí vận chuyển, hoặc các điều chỉnh kế toán), thay vì các mặt hàng mà khách hàng có thể mua.

Do chiếm tỷ trọng rất thấp và không mang thông tin hữu ích cho việc phân cụm khách hàng hay xây dựng hệ thống gợi ý sản phẩm, việc giữ lại các mã này có thể gây nhiễu và làm giảm chất lượng của mô hình.

Do đó, ta chỉ giữ lại những dòng không thuộc nhóm mã bất thường trong bộ dữ liệu.

```
In [18]: # Loại bỏ các dòng dữ liệu có mã sản phẩm thuộc nhóm bất thường
df = df[~df['StockCode'].isin(anomalous_stock_codes)]

# In ra số lượng dòng dữ liệu còn lại sau khi đã loại bỏ các mã sản phẩm bất thường
df.shape[0]
```

Out[18]: 399689

4.5 Xử lý cột mô tả sản phẩm (Description):

Sau khi loại bỏ các mã sản phẩm bất thường, các mô tả sản phẩm phản ánh chính xác hơn các mặt hàng thực tế được khách hàng mua, từ đó giúp hiểu rõ cấu trúc dữ liệu trước khi xây dựng mô hình gợi ý sản phẩm.

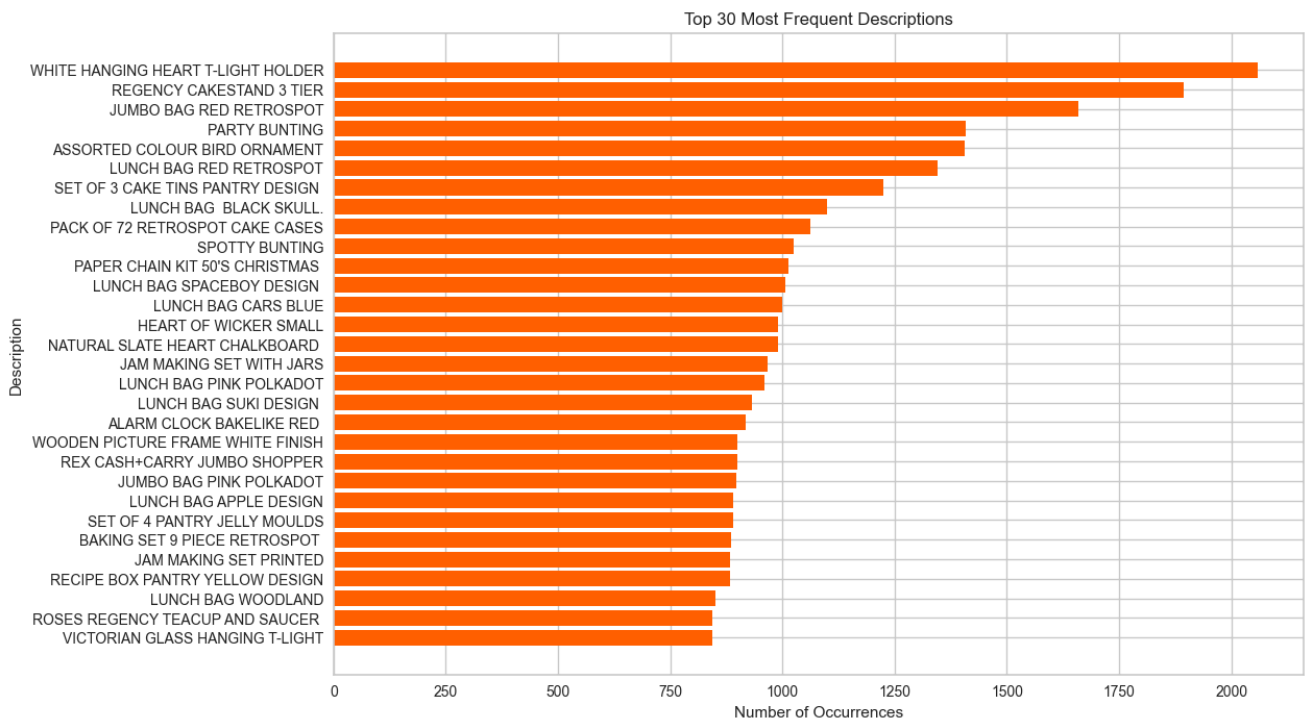
Những sản phẩm nào thu hút khách hàng mua nhiều nhất?

```
In [19]: # Đếm số lần xuất hiện của từng mô tả sản phẩm (Description)
description_counts = df['Description'].value_counts()

# Lấy ra 30 mô tả sản phẩm xuất hiện nhiều nhất
top_30_descriptions = description_counts[:30]

# Vẽ biểu đồ thanh ngang
plt.figure(figsize=(12,8))
plt.barh(top_30_descriptions.index[::-1], top_30_descriptions.values[::-1], color='#ff6200')

plt.xlabel('Number of Occurrences')
plt.ylabel('Description')
plt.title('Top 30 Most Frequent Descriptions')
plt.show()
```



Nhận xét:

- Các mô tả xuất hiện nhiều nhất thường là các vật dụng gia đình, đặc biệt là những món liên quan đến đồ dùng nhà bếp, túi đựng cơm trưa và các món đồ trang trí.

- Tất cả các mô tả đều được viết in hoa, điều này có thể là định dạng chuẩn khi nhập mô tả sản phẩm vào cơ sở dữ liệu.

Tuy nhiên, xem xét những bất thường và không nhất quán gặp phải trong bộ dữ liệu nên ta kiểm tra xem có các mô tả được nhập bằng chữ thường hoặc kết hợp các kiểu chữ hay không.

```
In [20]: # Lấy danh sách các mô tả sản phẩm (Description) duy nhất trong tập dữ liệu
lowercase_descriptions = df['Description'].unique()

# Lọc ra những mô tả có chứa ít nhất một ký tự chữ thường (a-z)
# any(char.islower() for char in desc): kiểm tra xem trong mô tả có tồn tại chữ thường hay không
lowercase_descriptions = [desc for desc in lowercase_descriptions if any(char.islower() for char in desc)]

# In ra danh sách
print("Các mô tả duy nhất có chứa ký tự chữ thường là:")
print("-"*60)
for desc in lowercase_descriptions:
    print(desc)
```

Các mô tả duy nhất có chứa ký tự chữ thường là:

```
-----
BAG 500g SWIRLY MARBLES
POLYESTER FILLER PAD 45x45cm
POLYESTER FILLER PAD 45x30cm
POLYESTER FILLER PAD 40x40cm
FRENCH BLUE METAL DOOR SIGN No
BAG 250g SWIRLY MARBLES
BAG 125g SWIRLY MARBLES
3 TRADITIONAL BISCUIT CUTTERS SET
NUMBER TILE COTTAGE GARDEN No
FOLK ART GREETING CARD,pack/12
ESSENTIAL BALM 3.5g TIN IN ENVELOPE
POLYESTER FILLER PAD 65CMx65CM
NUMBER TILE VINTAGE FONT No
POLYESTER FILLER PAD 30CMx30CM
POLYESTER FILLER PAD 60x40cm
FLOWERS HANDBAG blue and orange
Next Day Carriage
THE KING GIFT BAG 25x24x12cm
High Resolution Image
```

Nhận xét:

- Danh sách trên cho thấy một số mô tả sản phẩm không được chuẩn hóa hoàn toàn về định dạng chữ, cụ thể là có sự xuất hiện của chữ thường xen lẫn chữ in hoa (ví dụ: cm, No, blue and orange, pack/12, Next Day Carriage, High Resolution Image).
- Những mục này dường như không liên quan đến sản phẩm thực tế và có thể đại diện cho các loại thông tin hoặc chi tiết dịch vụ khác.

Chiến lược xử lý:

1. Loại bỏ các bản ghi có mô tả chứa thông tin liên quan đến dịch vụ (ví dụ: "Next Day Carriage", "High Resolution Image"), vì những mô tả này không đại diện cho sản phẩm vật lý thực tế.
2. Đối với các mô tả sản phẩm còn lại có sự pha trộn giữa chữ in hoa và chữ thường, tiến hành chuẩn hóa toàn bộ văn bản về chữ in hoa nhằm đảm bảo tính nhất quán trong dữ liệu.

```
In [21]: # Danh sách các mô tả không phải sản phẩm mà liên quan đến dịch vụ
service_related_descriptions = ["Next Day Carriage", "High Resolution Image"]

# Tính tỷ lệ (%) các dòng dữ liệu có mô tả thuộc nhóm dịch vụ
service_related_percentage = df[df['Description'].isin(service_related_descriptions)].shape[0] / df.shape[0]
```

```
print(f"Tỷ lệ các dòng có mô tả liên quan đến dịch vụ trong tập dữ liệu là: {service_related_percent}")

# Loại bỏ các dòng có thông tin Liên quan đến dịch vụ trong phần mô tả
df = df[~df['Description'].isin(service_related_descriptions)]

# Chuẩn hóa cột Description về chữ in hoa để đảm bảo tính đồng nhất
df['Description'] = df['Description'].str.upper()
```

Tỷ lệ các dòng có mô tả liên quan đến dịch vụ trong tập dữ liệu là: 0.02%

In [22]: `df.shape[0]`

Out[22]: 399606

Nhận xét:

- Tỷ lệ các mô tả liên quan đến dịch vụ chiếm một phần rất nhỏ (0.02%) trong tập dữ liệu. Do đó, việc loại bỏ các bản ghi này không làm ảnh hưởng đáng kể đến kích thước dữ liệu, đồng thời giúp tập dữ liệu tập trung hơn vào các giao dịch mua sản phẩm thực tế.
- Các mô tả dịch vụ còn lại (nếu có) xuất hiện với tần suất rất thấp ngoài nhóm các mô tả phổ biến nhất, nên tác động của chúng đến bài toán phân cụm khách hàng và gợi ý sản phẩm là không đáng kể.
- Hơn nữa, toàn bộ mô tả đã được chuẩn hóa về chữ in hoa, vì vậy nếu các mô tả này đại diện cho sản phẩm hợp lệ, chúng sẽ được gộp thống nhất vào đúng nhóm, tránh trùng lặp do khác biệt về kiểu chữ.

4.6 Xử lý cột giá sản phẩm (UnitPrice):

Trước tiên ta dùng `.describe()` sẽ xem xét mô tả thống kê của cột UnitPrice:

In [23]: `df['UnitPrice'].describe()`

```
Out[23]: count    399606.000000
mean         2.904957
std          4.448796
min           0.000000
25%          1.250000
50%          1.950000
75%          3.750000
max         649.500000
Name: UnitPrice, dtype: float64
```

Nhận xét:

- Giá trị đơn vị tối thiểu (min = 0): Điều này cho thấy có một số giao dịch mà giá đơn vị bằng không, có thể chỉ ra một món hàng miễn phí hoặc lỗi nhập dữ liệu.
- Để hiểu rõ bản chất của chúng, cần thiết phải điều tra kỹ hơn các giao dịch có giá đơn vị bằng không này.

Ta sẽ in thống kê mô tả cột số lượng giao dịch khi giá sản phẩm bằng không này.

In [24]: `df[df['UnitPrice']==0].describe()[['Quantity']]`

Out[24]:

Quantity	
count	33.000000
mean	420.515152
std	2176.713608
min	1.000000
25%	2.000000
50%	11.000000
75%	36.000000
max	12540.000000

Nhận xét:

- Các giao dịch có giá đơn vị bằng không tương đối ít về số lượng (33 giao dịch).
- Nhưng các giao dịch này có sự biến động lớn (std = 2176.713608) về số lượng sản phẩm.
- Min = 1.0: Tất cả 33 dòng này đều có số lượng dương. Nghĩa là đây không phải hàng trả lại hay điều chỉnh kho âm, mà thực sự là hàng được xuất đi.
- Max = 12,540: Có một giao dịch tặng/xuất hàng với số lượng lên tới hơn 12 ngàn sản phẩm nhưng giá vẫn bằng 0.

Nhận xét: Đây có thể là một đợt xả kho hàng lỗi hoặc lỗi nhập liệu nghiêm trọng ở một thời điểm nào đó.

Chiến lược xử lý

- Mặc dù số lượng giao dịch có đơn giá bằng không rất ít trong tổng số giao dịch, nhưng chúng có thể gây nhiễu và có khả năng làm lệch các mẫu hành vi khách hàng được xác định bởi thuật toán phân cụm.
- Chiến lược nên là loại bỏ các giao dịch này khỏi bộ dữ liệu. Việc này sẽ giúp duy trì một bộ dữ liệu sạch hơn và nhất quán hơn, điều này rất quan trọng để xây dựng một mô hình phân cụm chính xác và đáng tin cậy, cũng như hệ thống gợi ý.

Do đó, ta chỉ giữ lại những giao dịch có đơn giá dương (UnitPrice > 0)

```
In [25]: df = df[df['UnitPrice'] > 0]
```

4.7 Xử lý các giá trị ngoại lai:

- Thuật toán phân cụm K-means vốn nhạy cảm với quy mô dữ liệu và các giá trị ngoại lai (outliers), vì chúng có thể làm lệch vị trí các tâm cụm và gây sai số từ đó dẫn đến kết quả phân nhóm không chính xác.
- Tuy nhiên, ở giai đoạn hiện tại, dữ liệu vẫn đang ở cấp độ giao dịch (transaction-level). Một giao dịch có giá trị đột biến chưa hẳn là ngoại lai của một hành vi dài hạn của vi khách hàng, mà có thể chỉ là một sự kiện đơn lẻ. Việc loại bỏ sớm các quan sát này có nguy cơ làm mất đi các tín hiệu quan trọng cho việc định hình chân dung khách hàng.
- Do đó, việc xử lý ngoại lai sẽ được xử lý cho đến khi dữ liệu được tổng hợp thành các đặc trưng cấp độ khách hàng (customer-level features), đảm bảo tính toàn vẹn của thông tin đầu vào.

```
In [26]: # Đặt lại chỉ số (index) của DataFrame sau các bước lọc và Loại bỏ dữ liệu
df.reset_index(drop=True, inplace=True)
```

```
# Kiểm tra Lại số Lượng dòng dữ Liệu còn Lại trong tập dữ Liệu
df.shape[0]
```

Out[26]: 399573

5. XÂY DỰNG ĐẶC TRƯNG (Feature Engineering)

Mô tả: Chúng ta sẽ tạo ra 3 đặc trưng mới (RFM) cho mỗi khách hàng:

- Recency (R) - Gần đây: Số ngày kể từ lần mua hàng cuối cùng của khách đến thời điểm hiện tại. Giá trị recency thấp hơn có nghĩa là khách hàng đã mua hàng gần đây hơn.
- Frequency (F) - Tần suất: Tổng số lần khách đã thực hiện giao dịch trong một khoảng thời gian nhất định. Giá trị frequency cao hơn cho thấy khách hàng trung thành hoặc sự hài lòng cao hơn.
- Monetary (M) - Giá trị tiền tệ: Tổng số tiền khách đã chi tiêu trong một khoảng thời gian nhất định. Khách hàng có giá trị tiền tệ cao hơn đã đóng góp nhiều hơn cho doanh nghiệp, cho thấy tiềm năng giá trị lâu dài cao.

Kết hợp lại, các chỉ số này giúp hiểu hành vi mua hàng và sở thích của khách hàng, điều này rất quan trọng trong việc cá nhân hóa chiến lược marketing và tạo ra hệ thống gợi ý.

5.1 Recency (R)

```
In [27]: # 1. Chuyển cột InvoiceDate sang định dạng thời gian (datetime)
df['InvoiceDate'] = pd.to_datetime(df['InvoiceDate'])

# 2. Trích xuất ngày (không bao gồm giờ) từ InvoiceDate
df['InvoiceDay'] = df['InvoiceDate'].dt.date

# 3. Với mỗi khách hàng, xác định ngày mua hàng gần nhất
customer_data = df.groupby('CustomerID')['InvoiceDay'].max().reset_index()

# 4. Xác định ngày mua hàng gần nhất trong toàn bộ tập dữ Liệu
most_recent_date = df['InvoiceDay'].max()

# 5. Chuyển các giá trị ngày về định dạng datetime để tính toán chênh lệch thời gian
customer_data['InvoiceDay'] = pd.to_datetime(customer_data['InvoiceDay'])
most_recent_date = pd.to_datetime(most_recent_date)

# 6. Tính số ngày kể từ lần mua hàng gần nhất của mỗi khách hàng
# Days_Since_Last_Purchase càng nhỏ → khách hàng mua hàng càng gần đây
customer_data['Days_Since_Last_Purchase'] = (most_recent_date - customer_data['InvoiceDay']).dt.days

# 7. Loại bỏ cột InvoiceDay vì đã hoàn thành việc tính toán
customer_data.drop(columns=['InvoiceDay'], inplace=True)
```

```
In [28]: customer_data.head()
```

```
Out[28]:
```

	CustomerID	Days_Since_Last_Purchase
0	12346.0	325
1	12347.0	2
2	12348.0	75
3	12349.0	18
4	12350.0	310

5.2 Frequency (F)

```
In [29]: # 1. Tính tổng số giao dịch của mỗi khách hàng
# Mỗi mã hóa đơn (InvoiceNo) đại diện cho một lần mua hàng
total_transactions = df.groupby('CustomerID')['InvoiceNo'].nunique().reset_index()
total_transactions.rename(columns={'InvoiceNo': 'Total_Transactions'}, inplace=True)

# 2. Tính tổng số sản phẩm mà mỗi khách hàng đã mua
# Quantity phản ánh số Lượng sản phẩm trong toàn bộ lịch sử giao dịch
total_products_purchased = df.groupby('CustomerID')['Quantity'].sum().reset_index()
total_products_purchased.rename(columns={'Quantity': 'Total_Products_Purchased'}, inplace=True)

# 3. Gộp các chỉ số giao dịch vào bảng dữ liệu khách hàng
customer_data = pd.merge(customer_data, total_transactions, on='CustomerID')
customer_data = pd.merge(customer_data, total_products_purchased, on='CustomerID')
```

```
In [30]: customer_data.head()
```

```
Out[30]:
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased
0	12346.0	325	2	0
1	12347.0	2	7	2458
2	12348.0	75	4	2332
3	12349.0	18	1	630
4	12350.0	310	1	196

5.3 Monetary (M)

```
In [31]: # 1. Tạo cột Total_Spend: tổng tiền mỗi dòng giao dịch = đơn giá * số Lượng
df['Total_Spend'] = df['UnitPrice'] * df['Quantity']

# 2. Tính tổng số tiền mà mỗi khách hàng đã chi tiêu trong toàn bộ lịch sử mua hàng
total_spend = df.groupby('CustomerID')['Total_Spend'].sum().reset_index()

# 3. Ghép tổng số tiền đã chi tiêu với số Lượng giao dịch
# Tính giá trị trung bình mỗi giao dịch của khách hàng
average_transaction_value = total_spend.merge(df, on='CustomerID')
average_transaction_value['Average_Transaction_Value'] = average_transaction_value['Total_Spend'] /

# 4. Gộp các chỉ số vào bảng dữ liệu khách hàng
customer_data = pd.merge(customer_data, total_spend, on='CustomerID')
customer_data = pd.merge(customer_data, average_transaction_value[['CustomerID', 'Average_Transaction_Value']], on='CustomerID')
```

```
In [32]: customer_data.head()
```

```
Out[32]:
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_Transaction_Value
0	12346.0	325	2	0	0.00	0.00
1	12347.0	2	7	2458	4310.00	615.71
2	12348.0	75	4	2332	1437.24	359.31
3	12349.0	18	1	630	1457.55	1457.55
4	12350.0	310	1	196	294.40	294.40

5.4 Đa dạng sản phẩm (Product Diversity)

Chỉ số này dùng để đo lường mức độ đa dạng sản phẩm trong hành vi mua sắm của khách hàng.

Đặc điểm này thể hiện số lượng sản phẩm khác nhau mà khách hàng đã mua. Giá trị cao hơn cho thấy khách hàng có sở thích đa dạng, mua nhiều loại sản phẩm, trong khi giá trị thấp hơn có thể chỉ ra sở thích tập trung hoặc cụ thể.

```
In [33]: # 1. Tính số lượng sản phẩm duy nhất mà mỗi khách hàng đã mua
# 'StockCode' đại diện cho từng mã sản phẩm khác nhau
unique_products_purchased = df.groupby('CustomerID')['StockCode'].nunique().reset_index()
unique_products_purchased.rename(columns={'StockCode': 'Unique_Products_Purchased'}, inplace=True)

# 2. Gộp thông tin số lượng sản phẩm duy nhất vào bảng dữ liệu khách hàng
customer_data = pd.merge(customer_data, unique_products_purchased, on='CustomerID')
```

```
In [34]: customer_data.head()
```

```
Out[34]:
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average
0	12346.0	325	2	0	0.00	
1	12347.0	2	7	2458	4310.00	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.40	

5.5 Đặc điểm hành vi (Behavioral Features)

Đặc điểm này khai thác hành vi mua sắm theo thời gian của khách hàng thông qua các chỉ số:

Số ngày trung bình giữa các lần mua hàng:

- Chỉ số này cho biết khoảng thời gian trung bình giữa hai lần mua của một khách hàng. Qua đó có thể hiểu được tần suất mua sắm và dự đoán thời điểm khách hàng có khả năng quay lại mua hàng.

Ngày mua sắm ưa thích:

- Thể hiện ngày trong tuần mà khách hàng mua sắm nhiều nhất. Thông tin này giúp nhận biết thói quen mua sắm theo ngày của từng khách hàng hoặc nhóm khách hàng.

Giờ mua sắm ưa thích:

- Cho biết khung giờ trong ngày mà khách hàng thường mua sắm nhiều nhất. Nhờ đó có thể lựa chọn thời điểm phù hợp để triển khai các chương trình ưu đãi hoặc thông báo.

```
In [35]: # 1. Trích xuất ngày trong tuần và giờ mua sắm từ cột InvoiceDate
df['Day_Of_Week'] = df['InvoiceDate'].dt.dayofweek
df['Hour'] = df['InvoiceDate'].dt.hour

# 2. Tính khoảng cách (số ngày) giữa các lần mua liên tiếp của mỗi khách hàng
# Dùng diff() để tính khoảng cách ngày giữa các giao dịch liên tiếp
days_between_purchases = df.groupby('CustomerID')['InvoiceDay'].filter(lambda x: len(x) > 1) \
    .groupby(df['CustomerID']).apply(lambda x: x.diff().dt.days.dropna())

# 3. Tính trung bình số ngày giữa các lần mua hàng cho mỗi khách hàng
average_days_between_purchases = days_between_purchases.groupby('CustomerID').mean().reset_index()
average_days_between_purchases.rename(columns={'InvoiceDay': 'Average_Days_Between_Purchases'}, inplace=True)

# 4. Xác định ngày trong tuần mà khách hàng mua sắm nhiều nhất
```

```

favorite_shopping_day = df.groupby(['CustomerID', 'Day_Of_Week']).size().reset_index(name='Count')
favorite_shopping_day = favorite_shopping_day.loc[favorite_shopping_day.groupby('CustomerID')['Count']

# 5. Xác định giờ trong ngày mà khách hàng mua sắm nhiều nhất
favorite_shopping_hour = df.groupby(['CustomerID', 'Hour']).size().reset_index(name='Count')
favorite_shopping_hour = favorite_shopping_hour.loc[favorite_shopping_hour.groupby('CustomerID')['Co

# 6. Gộp các đặc trưng thời gian vào bảng dữ liệu khách hàng
customer_data = pd.merge(customer_data, average_days_between_purchases, on='CustomerID')
customer_data = pd.merge(customer_data, favorite_shopping_day, on='CustomerID')
customer_data = pd.merge(customer_data, favorite_shopping_hour, on='CustomerID')

```

In [36]: `customer_data.head()`

Out[36]:

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_
0	12346.0	325	2	0	0.00	
1	12347.0	2	7	2458	4310.00	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.40	

5.6 Đặc điểm địa lý (Geographic Features)

Việc hiểu được phân bố địa lý của khách hàng là rất quan trọng vì một số lý do:

Đặc điểm này xác định quốc gia mà mỗi khách hàng đang sinh sống có thể giúp chúng ta hiểu các mô hình mua hàng và sở thích đặc thù theo vùng.

Các vùng khác nhau có thể có sở thích và hành vi mua sắm khác nhau, điều này rất quan trọng trong việc cá nhân hóa chiến lược marketing và lập kế hoạch tồn kho.

Đầu tiên ta xem tỉ lệ phần trăm của từng quốc gia xuất hiện trong dataframe.

In [37]: `df['Country'].value_counts(normalize=True).head()`

Out[37]:

```

Country
United Kingdom    0.890971
Germany           0.022722
France            0.020402
EIRE              0.018440
Spain             0.006162
Name: proportion, dtype: float64

```

Nhận xét:

- Ta thấy rằng phần lớn giao dịch (khoảng 89%) thuộc về Vương quốc Anh (UK). Điều này cho thấy dữ liệu chủ yếu phản ánh hành vi của khách hàng UK, nên mô hình phân cụm sẽ chịu ảnh hưởng lớn từ nhóm này.
- Các quốc gia khác chiếm tỷ lệ rất nhỏ chỉ chiếm tổng cộng khoảng 11% các giao dịch.

Phương án:

- Tạo biến nhị phân `Is_UK` để tách khách UK với khách nước ngoài, giúp đơn giản hóa phân tích và mô hình hóa.
- Với các quốc gia có tỷ lệ quá nhỏ, có thể gộp thành nhóm "Other" để tránh quá nhiều nhãn rời rạc.

```
In [38]: # 1. Nhóm dữ liệu theo CustomerID và Country, đếm số Lượng giao dịch của mỗi khách hàng trong từng q
customer_country = df.groupby(['CustomerID', 'Country']).size().reset_index(name='Number_of_Transact

# 2. Sắp xếp theo số Lượng giao dịch giảm dần và lấy quốc gia chính của mỗi khách hàng
# (drop_duplicates theo 'CustomerID' giữ lại dòng đầu tiên, tức là quốc gia có nhiều giao dịch nhất)
customer_main_country = customer_country.sort_values('Number_of_Transactions', ascending=False).drop

# 3. Tạo cột mới 'Is_UK': 1 nếu quốc gia chính là 'United Kingdom', 0 nếu không phải
customer_main_country['Is_UK'] = customer_main_country['Country'].apply(lambda x: 1 if x == 'United

# 4. Gộp thông tin cột 'Is_UK' vào bảng dữ liệu khách hàng
customer_data = pd.merge(customer_data, customer_main_country[['CustomerID', 'Is_UK']], on='Customer

# 5. Xem 5 dòng đầu tiên
customer_data.head()
```

```
Out[38]:
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average
0	12346.0	325	2	0	0.00	
1	12347.0	2	7	2458	4310.00	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.40	

```
In [39]: # Đếm số Lượng khách hàng ở UK và không ở UK
customer_data['Is_UK'].value_counts()
```

```
Out[39]: Is_UK
1      3866
0       416
Name: count, dtype: int64
```

5.7 Hành vi hủy đơn (Cancellation Insights)

Trong thương mại điện tử, đây là chỉ số giúp nhận diện các nhóm khách hàng rủi ro hoặc các vấn đề về vận hành.

Tần suất hủy:

- Chỉ số này đại diện cho tổng số giao dịch mà khách hàng đã hủy (tần suất hủy) .
- Đây có thể là dấu hiệu của sự không hài lòng hoặc các vấn đề khác, và việc hiểu điều này có thể giúp chúng ta điều chỉnh các chiến lược nhằm giảm số lần hủy và nâng cao sự hài lòng của khách hàng.

Tỷ lệ hủy:

- Tỷ lệ hủy cao có thể là dấu hiệu của một phân khúc khách hàng chưa hài lòng. Bằng cách xác định các phân khúc này, chúng ta có thể phát triển các chiến lược hướng tới nhằm cải thiện trải nghiệm mua sắm của họ và tiềm năng giảm tỷ lệ hủy đơn.

```
In [40]: # 1. Tính tổng số giao dịch của mỗi khách hàng
total_transactions = df.groupby('CustomerID')['InvoiceNo'].nunique().reset_index()

# 2. Lọc các giao dịch có trạng thái hủy và tính số lần hủy của mỗi khách hàng
cancelled_transactions = df[df['Transaction_Status'] == 'Cancelled']
cancellation_frequency = cancelled_transactions.groupby('CustomerID')['InvoiceNo'].nunique().reset_i
cancellation_frequency.rename(columns={'InvoiceNo': 'Cancellation_Frequency'}, inplace=True)
```

```
# 3. Gộp thông tin số lần hủy vào bảng dữ liệu khách hàng
# Dùng (how='left') để giữ tất cả khách hàng, kể cả những người chưa hủy giao dịch
customer_data = pd.merge(customer_data, cancellation_frequency, on='CustomerID', how='left')

# 4. Thay giá trị NaN (khách hàng không có giao dịch hủy) bằng 0
customer_data.fillna({'Cancellation_Frequency': 0}, inplace=True)

# 5. Tính tỉ lệ hủy (Cancellation Rate) = Số lần hủy / Tổng số giao dịch
customer_data['Cancellation_Rate'] = customer_data['Cancellation_Frequency'] / total_transactions['1

# 6. Xem 5 dòng đầu tiên
customer_data.head()
```

Out[40]:

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_
0	12346.0	325	2	0	0.00	
1	12347.0	2	7	2458	4310.00	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.40	

5.8 Tính biến động và xu hướng chi tiêu (Seasonality & Trends)

Đặc trưng này giúp mức độ thay đổi trong hành vi mua sắm của khách hàng theo thời gian, từ đó hỗ trợ dự đoán hành vi mua sắm trong tương lai.

Monthly_Spending_Mean:

- Đây là số tiền trung bình mà một khách hàng chi tiêu hàng tháng. Nó giúp chúng ta đánh giá thói quen chi tiêu chung của từng khách hàng.
- Giá trị trung bình cao cho thấy khách hàng chi tiêu nhiều hơn, có khả năng quan tâm đến các sản phẩm cao cấp, trong khi giá trị trung bình thấp có thể chỉ ra khách hàng tiết kiệm hơn.

Monthly_Spending_Std:

- Giá trị này cho biết độ biến động trong chi tiêu hàng tháng của khách hàng.
- Giá trị cao cho thấy chi tiêu của khách hàng dao động đáng kể từ tháng này sang tháng khác, có thể là do các khoản mua lớn không đều.
- Ngược lại, giá trị thấp cho thấy thói quen chi tiêu ổn định và nhất quán hơn.
- Độ biến động này có thể hỗ trợ trong việc thiết kế các chương trình khuyến mãi hoặc giảm giá cá nhân hóa trong những khoảng thời gian họ được dự đoán sẽ chi tiêu nhiều hơn.

Spending_Trend:

- Xu hướng chi tiêu của khách hàng theo thời gian được tính bằng độ dốc của đường xu hướng tuyến tính phù hợp với dữ liệu chi tiêu của họ.
- Giá trị dương chỉ xu hướng chi tiêu tăng, có thể chỉ ra lòng trung thành hoặc sự hài lòng gia tăng.
- Ngược lại, xu hướng âm có thể báo hiệu sự quan tâm hoặc hài lòng giảm, nhấn mạnh nhu cầu về các chiến lược tái tương tác.
- Giá trị gần bằng không cho thấy thói quen chi tiêu ổn định.

```
In [41]: # 1. Tách năm và tháng từ cột InvoiceDate
df['Year'] = df['InvoiceDate'].dt.year
df['Month'] = df['InvoiceDate'].dt.month

# 2. Tính tổng chi tiêu mỗi khách hàng theo từng tháng
# Nhóm theo CustomerID, Year, Month và tính tổng cột Total_Spend
monthly_spending = df.groupby(['CustomerID', 'Year', 'Month'])['Total_Spend'].sum().reset_index()

# 3. Tính các chỉ số chi tiêu theo mùa cho từng khách hàng
# mean: trung bình chi tiêu hàng tháng
# std: độ lệch chuẩn chi tiêu hàng tháng (biến động)
seasonal_buying_patterns = monthly_spending.groupby('CustomerID')['Total_Spend'].agg(['mean', 'std'])
seasonal_buying_patterns.rename(columns={'mean': 'Monthly_Spending_Mean', 'std': 'Monthly_Spending_Std'})

# 4. Điền giá trị thiếu nếu khách hàng chỉ có một tháng chi tiêu → std sẽ là NaN, thay bằng 0
seasonal_buying_patterns['Monthly_Spending_Std'] = seasonal_buying_patterns['Monthly_Spending_Std'].fillna(0)

# 5. Định nghĩa hàm tính xu hướng chi tiêu theo thời gian
def calculate_trend(spend_data):
    """
    Nhận vào một chuỗi chi tiêu theo tháng của một khách hàng,
    trả về slope của hồi quy tuyến tính (xu hướng tăng/giảm chi tiêu)
    """

    if len(spend_data) > 1:
        x = np.arange(len(spend_data))
        slope, _, _, _ = linregress(x, spend_data)
        return slope

    else:
        return 0 # nếu chỉ có 1 tháng, slope = 0

# 6. Tính xu hướng chi tiêu (slope) cho từng khách hàng
spending_trends = monthly_spending.groupby('CustomerID')['Total_Spend'].apply(calculate_trend).reset_index()
spending_trends.rename(columns={'Total_Spend': 'Spending_Trend'}, inplace=True)

# 7. Gộp các thông tin chi tiêu theo mùa và xu hướng vào bảng dữ liệu khách hàng
customer_data = pd.merge(customer_data, seasonal_buying_patterns, on='CustomerID')
customer_data = pd.merge(customer_data, spending_trends, on='CustomerID')

# 8. Xem 5 dòng đầu tiên
customer_data.head()
```

```
Out[41]:
```

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average
--	------------	--------------------------	--------------------	--------------------------	-------------	---------

0	12346.0	325	2	0	0.00	
1	12347.0	2	7	2458	4310.00	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.40	



5.9 Tổng quan dữ liệu sau khi xây dựng các đặc trưng cho mỗi khách hàng

Xử lý kiểu dữ liệu:

- Chuyển kiểu dữ liệu của CustomerID sang string `.astype(str)` Vì CustomerID là mã định danh duy nhất, không dùng để tính toán.

- Dùng `convert_dtypes()` để tự động chuyển các cột về kiểu dữ liệu phù hợp: số nguyên (Int64), số thực (Float64), hoặc chuỗi (string).
- Việc này giúp tiết kiệm bộ nhớ và chuẩn hóa kiểu dữ liệu cho các bước xử lý tiếp theo.

```
In [42]: customer_data['CustomerID'] = customer_data['CustomerID'].astype(str)
customer_data = customer_data.convert_dtypes()
customer_data.head(10)
```

Out[42]:

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_
0	12346.0	325	2	0	0.0	
1	12347.0	2	7	2458	4310.0	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.4	
5	12352.0	36	8	463	1265.41	
6	12353.0	204	1	20	89.0	
7	12354.0	232	1	530	1079.4	
8	12355.0	214	1	240	459.4	
9	12356.0	22	3	1573	2487.43	

```
In [43]: customer_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4282 entries, 0 to 4281
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   CustomerID                            4282 non-null   string
1   Days_Since_Last_Purchase               4282 non-null   Int64
2   Total_Transactions                     4282 non-null   Int64
3   Total_Products_Purchased               4282 non-null   Int64
4   Total_Spend                            4282 non-null   Float64
5   Average_Transaction_Value              4282 non-null   Float64
6   Unique_Products_Purchased              4282 non-null   Int64
7   Average_Days_Between_Purchases         4282 non-null   Float64
8   Day_Of_Week                            4282 non-null   Int32
9   Hour                                    4282 non-null   Int32
10  Is_UK                                   4282 non-null   Int64
11  Cancellation_Frequency                 4282 non-null   Int64
12  Cancellation_Rate                      4282 non-null   Float64
13  Monthly_Spending_Mean                  4282 non-null   Float64
14  Monthly_Spending_Std                    4282 non-null   Float64
15  Spending_Trend                         4282 non-null   Float64
dtypes: Float64(7), Int32(2), Int64(6), string(1)
memory usage: 564.7 KB
```

Ý nghĩa của các cột mới tạo trong dataframe customer_data

1. CustomerID: Mã định danh duy nhất của mỗi khách hàng.
2. Days_Since_Last_Purchase: Số ngày kể từ lần mua gần nhất của khách hàng.
3. Total_Transactions: Tổng số giao dịch khách hàng đã thực hiện.
4. Total_Products_Purchased: Tổng số lượng sản phẩm đã mua.

5. Total_Spend: Tổng số tiền khách hàng đã chi tiêu.
6. Average_Transaction_Value: Giá trị trung bình mỗi giao dịch.
7. Unique_Products_Purchased: Số loại sản phẩm khác nhau đã mua.
8. Average_Days_Between_Purchases: Số ngày trung bình giữa các lần mua.
9. Day_Of_Week: Ngày trong tuần khách hàng thường mua nhất (0 = Thứ Hai, 6 = Chủ Nhật).
10. Hour: Giờ trong ngày khách hàng thường mua nhất (0–23).
11. Is_UK: Biến nhị phân cho biết khách hàng có thuộc UK hay không (1 = UK, 0 = không).
12. Cancellation_Frequency: Số lần khách hàng hủy giao dịch.
13. Cancellation_Rate: Tỷ lệ giao dịch bị hủy trên tổng số giao dịch.
14. Monthly_Spending_Mean: Chi tiêu trung bình mỗi tháng của khách hàng.
15. Monthly_Spending_Std: Mức độ biến động chi tiêu giữa các tháng.
16. Spending_Trend: Xu hướng chi tiêu theo thời gian (tăng, giảm hoặc ổn định).

6. XỬ LÝ GIÁ TRỊ NGOẠI LẠI

Ta sử dụng thuật toán Isolation Forest là một kỹ thuật Machine Learning mạnh mẽ để loại bỏ những khách hàng có hành vi bất thường (ví dụ: chi tiêu quá khủng khiếp, tỷ lệ hủy đơn 100%, hoặc mua hàng với số lượng cực lớn) trước khi tiến hành phân cụm.

Hoạt động: Isolation Forest cô lập điểm bất thường thông qua các phép chia ngẫu nhiên. Điểm nào bị cô lập với ít lần chia sẽ được xem là outlier.

Isolation Forest phát hiện khách hàng bất thường dựa trên việc 'cô lập' họ trong không gian nhiều chiều của các đặc trưng hành vi. Những khách hàng này sẽ bị cô lập nhanh và được gán nhãn ngoại lai.

Giả định khoảng 5% khách hàng là bất thường (contamination=0.05).

```
In [44]: # 1. Khởi tạo mô hình Isolation Forest
# contamination=0.05 → giả định 5% dữ liệu là ngoại lai (outliers)
# random_state=0 → để kết quả cố định mỗi lần chạy lặp lại
model = IsolationForest(contamination=0.05, random_state=0)

# 2. Dự đoán điểm ngoại lai
# .fit_predict() vừa học mô hình vừa trả về nhãn dự đoán cho từng khách hàng
# .iloc[:, 1:] → Loại bỏ cột CustomerID, chỉ dùng các cột số để phân tích
# Kết quả trả về là mảng: -1 → ngoại lai, 1 → bình thường
customer_data['Outlier_Scores'] = model.fit_predict(customer_data.iloc[:, 1:].to_numpy())

# 3. Tạo biến nhị phân Is_Outlier
# 1 → ngoại lai, 0 → bình thường
customer_data['Is_Outlier'] = [1 if x == -1 else 0 for x in customer_data['Outlier_Scores']]

# 4. Xem 5 dòng đầu tiên của bảng dữ liệu
customer_data.head()
```

Out[44]:

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average_
0	12346.0	325	2	0	0.0	
1	12347.0	2	7	2458	4310.0	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.4	

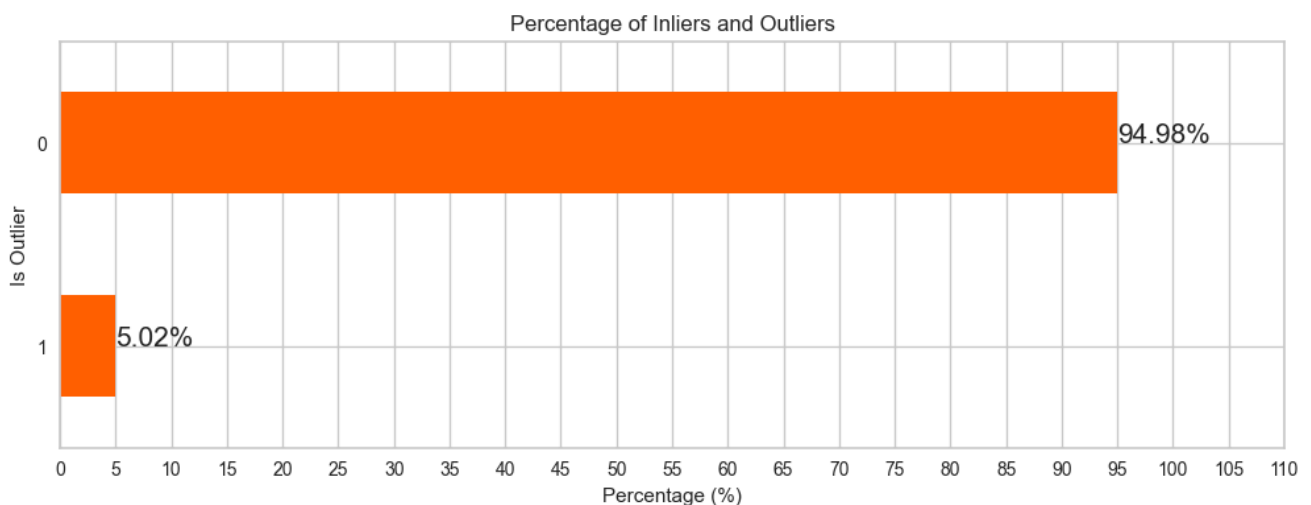
Trong toàn bộ cơ sở dữ liệu, có bao nhiêu phần trăm khách hàng là 'bất thường' và bao nhiêu là 'bình thường'?

```
In [45]: # 1. Tính tỉ lệ phần trăm inliers và outliers
outlier_percentage = customer_data['Is_Outlier'].value_counts(normalize=True) * 100

# 2. Vẽ biểu đồ thanh ngang
plt.figure(figsize=(12, 4))
outlier_percentage.plot(kind='barh', color='#ff6200')

for index, value in enumerate(outlier_percentage):
    plt.text(value, index, f'{value:.2f}%', fontsize=15)

plt.title('Percentage of Inliers and Outliers')
plt.xticks(ticks=np.arange(0, 115, 5))
plt.xlabel('Percentage (%)')
plt.ylabel('Is Outlier')
plt.gca().invert_yaxis() # Đảo chiều trục Y
plt.show()
```



Nhận xét:

- Is_Outlier = 0 (Bình thường): Chiếm khoảng 95.00%. Đây là nhóm khách hàng có hành vi mua sắm phổ biến, ổn định, dùng để đại diện cho số đông khi bạn thực hiện phân cụm.
- Is_Outlier = 1 (Bất thường): Chiếm khoảng 5.00%. Đây là những khách hàng có doanh thu cực khủng hoặc tỷ lệ hủy đơn quá cao,... mà thuật toán Isolation Forest đã lọc ra dựa trên tham số contamination=0.05.

Phương án xử lý ngoại lai:

- Vì mục tiêu dự án là sử dụng kỹ thuật phân cụm (Clustering), việc xử lý các giá trị ngoại lai này là rất quan trọng để ngăn chúng ảnh hưởng đáng kể đến chất lượng các cụm.

- Do đó, ta sẽ tách các giá trị ngoại lai này ra để phân tích thêm và loại bỏ chúng khỏi tập dữ liệu chính nhằm chuẩn bị cho phân tích phân cụm.

```
In [46]: # 1. Lọc các khách hàng được đánh dấu là outliers (Is_Outlier == 1)
outliers_data = customer_data[customer_data['Is_Outlier'] == 1]

# 2. Lấy các khách hàng bình thường (không phải ngoại lai)
customer_data_cleaned = customer_data[customer_data['Is_Outlier'] == 0]

# 3. Loại bỏ các cột liên quan đến đánh dấu ngoại lai
customer_data_cleaned = customer_data_cleaned.drop(columns=['Outlier_Scores', 'Is_Outlier'])

# 4. Đặt lại chỉ số (index) của dataframe
customer_data_cleaned.reset_index(drop=True, inplace=True)
```

```
In [47]: customer_data_cleaned.shape[0]
```

```
Out[47]: 4067
```

Đây là số lượng khách hàng có hành vi bình thường được sử dụng cho bước phân cụm tiếp theo.

7. CHUẨN HÓA ĐẶC TRƯNG (Feature Scaling)

Tại sao phải chuẩn hóa các đặc trưng?

- Đối với K-means: Thuật toán dựa trên khoảng cách giữa các điểm dữ liệu, nên nếu các đặc trưng có đơn vị hoặc giá trị chênh lệch quá lớn, đặc trưng có giá trị lớn sẽ chi phối kết quả, làm cụm bị sai lệch.
- Đối với PCA: PCA tìm ra hướng có phương sai lớn nhất của dữ liệu. Nếu không chuẩn hóa, các đặc trưng có giá trị lớn sẽ chiếm ưu thế và làm sai lệch các thành phần chính.

Phương pháp:

Để đảm bảo các đặc trưng có ảnh hưởng cân bằng lên mô hình và phản ánh đúng cấu trúc dữ liệu, ta tiến hành chuẩn hóa (standardization) các đặc trưng. Việc chuẩn hóa chuyển đổi dữ liệu về phân phối có mean = 0 và standard deviation = 1, phù hợp cho các phương pháp dựa trên khoảng cách như PCA và K-means.

Tuy nhiên, không phải tất cả các đặc trưng đều cần chuẩn hóa. Các đặc trưng sau được loại trừ vì lý do cụ thể sau:

- **CustomerID**: Chỉ là mã định danh, không mang thông tin phục vụ phân cụm.
- **Is_UK**: Là biến nhị phân (0,1), việc chuẩn hóa không tạo khác biệt đáng kể.
- **Day_Of_Week**: Là biến phân loại dạng số (1–7), không mang tính chất liên tục nên không cần chuẩn hóa.

Chúng ta sẽ tiếp tục scaling các đặc trưng còn lại trong tập dữ liệu để chuẩn bị cho PCA và phân cụm K-means.

```
In [48]: # 1. Khởi tạo bộ chuẩn hóa (StandardScaler)
scaler = StandardScaler()

# 2. Danh sách các cột không cần scale
columns_to_exclude = ['CustomerID', 'Is_UK', 'Day_Of_Week']

# 3. Xác định danh sách các cột cần được scaling
columns_to_scale = customer_data_cleaned.columns.difference(columns_to_exclude)

# 4. Tạo bản sao của tập dữ liệu đã làm sạch
customer_data_scaled = customer_data_cleaned.copy()
```

```
# 5. Thực hiện chuẩn hóa trên các cột cần thiết
# (Hàm fit_transform sẽ tính toán trung bình/độ lệch chuẩn và biến đổi dữ liệu)
customer_data_scaled[columns_to_scale] = scaler.fit_transform(customer_data_scaled[columns_to_scale])

# 6. Hiển thị vài dòng đầu tiên của dữ liệu sau khi đã chuẩn hóa để kiểm tra
customer_data_scaled.head()
```

Out[48]:

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average
0	12346.0	2.345802	-0.477589	-0.754491	-0.813464	
1	12347.0	-0.905575	0.707930	2.005048	2.366920	
2	12348.0	-0.170744	-0.003381	1.863591	0.247087	
3	12349.0	-0.744516	-0.714692	-0.047205	0.262074	
4	12350.0	2.194809	-0.714692	-0.534446	-0.596223	

8. GIẢM CHIỀU DỮ LIỆU

Tại sao chúng ta cần giảm chiều dữ liệu (Dimensionality Reduction) ?

- Đa cộng tuyến (Multicollinearity Detected): Trong các bước trước, chúng ta có thể thấy rằng tập dữ liệu có chứa các đặc trưng đa cộng tuyến. Giảm chiều dữ liệu có thể giúp loại bỏ các thông tin dư thừa và giải quyết vấn đề này.
- Phân cụm tốt hơn với K-means: Vì K-means là một thuật toán dựa trên khoảng cách, việc có quá nhiều đặc trưng đôi khi có thể làm loãng (dilute) các quy luật ý nghĩa tiềm ẩn trong dữ liệu. Bằng cách giảm số chiều, chúng ta giúp K-means tìm ra các cụm chặt chẽ (compact) và tách biệt rõ ràng hơn.
- Giảm nhiễu (Noise Reduction): Bằng cách chỉ tập trung vào các đặc trưng quan trọng nhất, chúng ta có thể loại bỏ nhiễu trong dữ liệu, dẫn đến các cụm chính xác và ổn định hơn.
- Tăng cường khả năng trực quan hóa: Trong bối cảnh phân khúc khách hàng, việc có thể nhìn thấy các nhóm khách hàng trong không gian 2 hoặc 3 chiều mang lại những hiểu biết trực quan. Các kỹ thuật giảm chiều hỗ trợ điều này bằng cách rút gọn dữ liệu về một vài thành phần chính (principal components) để dễ dàng vẽ biểu đồ.
- Cải thiện hiệu quả tính toán: Giảm số lượng đặc trưng giúp tăng tốc độ tính toán trong quá trình mô hình hóa, làm cho thuật toán phân cụm hoạt động hiệu quả hơn.

Ở đây, chúng ta sẽ áp dụng PCA (Principal Components) lên tất cả các thành phần có sẵn và vẽ biểu đồ phương sai tích lũy (cumulative variance) được giải thích bởi chúng. PCA tạo ra các trục mới (PCs) gọi là thành phần chính, mỗi thành phần giữ một lượng thông tin nhất định của dữ liệu. "Thông tin" ở đây được đo bằng phương sai. PC1 luôn giữ nhiều thông tin nhất, sau đó đến PC2, PC3,... Chúng ta chọn số lượng PC sao cho tổng phương sai được giải thích nằm trong ngưỡng chấp nhận, ví dụ 80%.

In [49]:

```
# Thiết lập CustomerID làm cột chỉ mục (index) để không đưa vào tính toán
customer_data_scaled.set_index('CustomerID', inplace=True)

# Áp dụng thuật toán PCA
pca = PCA().fit(customer_data_scaled)

# Tính tổng tích lũy của phương sai được giải thích (Cumulative Explained Variance)
explained_variance_ratio = pca.explained_variance_ratio_
cumulative_explained_variance = np.cumsum(explained_variance_ratio)
```

```

# Thiết lập giá trị k tối ưu (dựa trên phân tích , ta chọn 6 thành phần)
optimal_k = 6

# Thiết lập phong cách hiển thị cho biểu đồ seaborn
sns.set(rc={'axes.facecolor': '#fcf0dc'}, style='darkgrid')

# Khởi tạo khung hình để vẽ biểu đồ
plt.figure(figsize=(20, 10))

# Vẽ biểu đồ cột thể hiện phương sai của từng thành phần riêng lẻ
barplot = sns.barplot(x=list(range(1, len(cumulative_explained_variance) + 1)),
                      y=explained_variance_ratio,
                      color='#fcc36d',
                      alpha=0.8)

# Vẽ biểu đồ đường thể hiện phương sai tích lũy (cộng dồn)
lineplot, = plt.plot(range(0, len(cumulative_explained_variance)), cumulative_explained_variance,
                     marker='o', linestyle='--', color='#ff6200', linewidth=2)

# Vẽ đường dọc màu đỏ đánh dấu vị trí k tối ưu đã chọn
optimal_k_line = plt.axvline(optimal_k - 1, color='red', linestyle='--', label=f'Giá trị k tối ưu = {optimal_k}')

# Đặt nhãn cho các trục và tiêu đề biểu đồ
plt.xlabel('Số lượng thành phần (Components)', fontsize=14)
plt.ylabel('Phương sai được giải thích (Explained Variance)', fontsize=14)
plt.title('Phương sai tích lũy vs. Số lượng thành phần', fontsize=18)

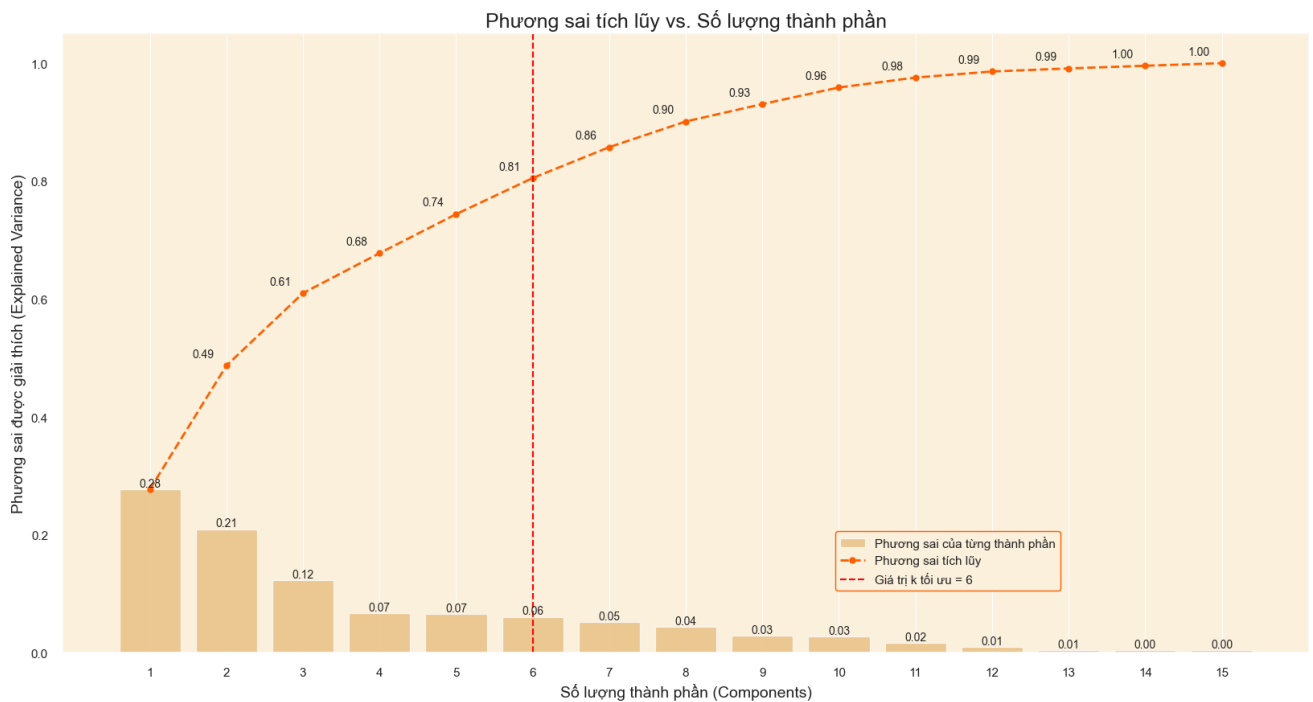
# Tùy chỉnh các vạch chia trục (ticks) và bảng chú giải (Legend)
plt.xticks(range(0, len(cumulative_explained_variance)))
plt.legend(handles=[barplot.patches[0], lineplot, optimal_k_line],
           labels=['Phương sai của từng thành phần', 'Phương sai tích lũy', f'Giá trị k tối ưu = {optimal_k}'],
           loc=(0.62, 0.1),
           frameon=True,
           framealpha=1.0,
           edgecolor='#ff6200')

# Hiển thị giá trị số cụ thể lên trên các cột và đường
x_offset = -0.3
y_offset = 0.01
for i, (ev_ratio, cum_ev_ratio) in enumerate(zip(explained_variance_ratio, cumulative_explained_variance)):
    # Ghi số liệu cho cột (phương sai riêng lẻ)
    plt.text(i, ev_ratio, f"{ev_ratio:.2f}", ha="center", va="bottom", fontsize=10)

    # Ghi số liệu cho đường (phương sai tích lũy) - bỏ qua điểm đầu tiên để tránh chồng chéo nếu cần
    if i > 0:
        plt.text(i + x_offset, cum_ev_ratio + y_offset, f"{cum_ev_ratio:.2f}", ha="center", va="bottom", fontsize=10)

plt.grid(axis='both')
plt.show()

```



Kết luận

Biểu đồ và các giá trị phương sai tích lũy được giải thích chỉ ra lượng tổng phương sai trong tập dữ liệu được nắm bắt bởi mỗi thành phần chính, cũng như phương sai tích lũy được giải thích bởi n thành phần đầu tiên.

Tại đây, chúng ta có thể quan sát thấy rằng:

- Thành phần đầu tiên giải thích khoảng 28% phương sai.
- Hai thành phần đầu tiên cùng nhau giải thích khoảng 49% phương sai.
- Ba thành phần đầu tiên giải thích khoảng 61% phương sai, và cứ tiếp tục như vậy.

Để chọn số lượng thành phần tối ưu, chúng ta thường tìm kiếm một điểm mà tại đó việc thêm một thành phần nữa không làm tăng đáng kể phương sai tích lũy, điểm này thường được gọi là "điểm khuỷu tay" (elbow point) trên đường cong.

Từ biểu đồ, chúng ta có thể thấy sự gia tăng phương sai tích lũy bắt đầu chậm lại sau thành phần thứ 6 (thành phần này giúp nắm bắt khoảng 81% tổng phương sai).

Xét trong bối cảnh phân khúc khách hàng, chúng ta muốn giữ lại đủ lượng thông tin cần thiết để xác định các nhóm khách hàng riêng biệt một cách hiệu quả. Do đó, việc giữ lại 6 thành phần đầu tiên có thể là một lựa chọn cân bằng, vì chúng cùng nhau giải thích một phần đáng kể tổng phương sai trong khi vẫn giảm được số chiều của tập dữ liệu.

```
In [50]: # 1. Khởi tạo đối tượng PCA với 6 thành phần (như đã phân tích ở bước trước)
pca = PCA(n_components=6)

# 2. Huấn luyện (fit) và biến đổi (transform) dữ liệu gốc sang hệ tọa độ PCA mới
customer_data_pca = pca.fit_transform(customer_data_scaled)

# 3. Tạo một DataFrame mới từ kết quả PCA, đặt tên các cột là PC1, PC2, v.v.
customer_data_pca = pd.DataFrame(customer_data_pca, columns=['PC'+str(i+1) for i in range(pca.n_components)])

# 4. Gán lại chỉ mục (CustomerID) từ dữ liệu gốc vào DataFrame PCA mới
customer_data_pca.index = customer_data_scaled.index

# Hiển thị kết quả để kiểm tra
customer_data_pca.head()
```

Out[50]:

	PC1	PC2	PC3	PC4	PC5	PC6
CustomerID						
12346.0	-2.186469	-1.705370	-1.576745	-1.008187	-0.411803	-1.658012
12347.0	3.290264	-1.387375	1.923310	0.930990	-0.010591	0.873150
12348.0	0.584684	0.585019	0.664727	0.655411	-0.470280	2.306657
12349.0	1.791116	-2.695652	5.850040	-0.853418	0.677111	-1.520098
12350.0	-1.997139	-0.542639	0.578781	-0.183682	-1.484838	0.062672

9. K-MEANS CLUSTERING

K-Means là một thuật toán học máy không giám sát (unsupervised machine learning) giúp phân nhóm dữ liệu thành một số lượng nhóm xác định trước (K) bằng cách tối thiểu hóa tổng bình phương sai số trong cụm (WCSS), hay còn gọi là quán tính (inertia).

Thuật toán thực hiện lặp đi lặp lại việc gán từng điểm dữ liệu vào tâm cụm (centroid) gần nhất, sau đó cập nhật lại vị trí các tâm cụm bằng cách tính trung bình cộng của tất cả các điểm đã được gán vào cụm đó. Quá trình này lặp lại cho đến khi đạt trạng thái hội tụ (convergence) hoặc chạm đến tiêu chuẩn dừng.

Hạn chế của thuật toán K-means và các hướng khắc phục:

- **Ảnh hưởng bởi số chiều dữ liệu (Curse of Dimensionality):** Trong không gian nhiều chiều, khoảng cách Euclidean trở nên kém ý nghĩa, làm tăng quán tính và giảm chất lượng phân cụm.

Áp dụng giảm chiều (PCA) trước khi phân cụm.

- **Nhạy cảm với điểm khởi tạo tâm cụm:** K-means có thể hội tụ vào nghiệm cục bộ tùy vào vị trí khởi tạo.

Khắc phục: Dùng k-means++ để khởi tạo thông minh.

- **Cần biết trước số cụm K:** K-means yêu cầu xác định số cụm trước khi chạy.

Khắc phục: Dùng Elbow method hoặc Silhouette analysis để ước lượng K tối ưu.

- **Không phù hợp với cụm không đều về kích thước/mật độ:** Các cụm thưa, dày hoặc không cân bằng dễ làm K-means phân cụm sai.

Khắc phục: Tăng n_init, hoặc đổi sang thuật toán phù hợp hơn như GMM hoặc DBSCAN.

- **Giả định cụm có dạng cầu và phương sai tương đồng:** K-means hoạt động kém với cụm kéo dài, phi tuyến hoặc dạng bất thường.

Khắc phục: Dùng thuật toán không dựa trên giả định này như DBSCAN hoặc GMM.

PCA đã được áp dụng vào tập dữ liệu từ trước để giảm chiều dữ liệu.

Đối với thuật toán K-Means, thiết lập tham số init là `k-means++` và tham số `n_init` là 10.

Để xác định số lượng cụm tối ưu, chúng ta sử dụng phương pháp khuỷu tay (elbow method) và phân tích hình bóng (silhouette analysis).

9.1 Xác định số lượng cụm tối ưu

Để xác định số lượng cụm tối ưu (k) cho việc phân khúc khách hàng, chúng ta sẽ khám phá hai phương pháp nổi tiếng:

- Elbow Method
- Silhouette Method

Việc sử dụng kết hợp cả hai phương pháp trong thực tế là rất phổ biến để kiểm chứng (corroborate) kết quả lẫn nhau.

Trong phần này, thư viện YellowBrick sẽ được sử dụng để hỗ trợ việc thực hiện phương pháp Khuỷu tay (Elbow method)

In [51]:

```
# 1. Thiết lập phong cách biểu đồ và màu nền (màu kem nhạt)
sns.set(style='darkgrid', rc={'axes.facecolor': '#fcf0dc'})

# 2. Thiết lập bảng màu cho biểu đồ (màu cam chủ đạo)
sns.set_palette(['#ff6200'])

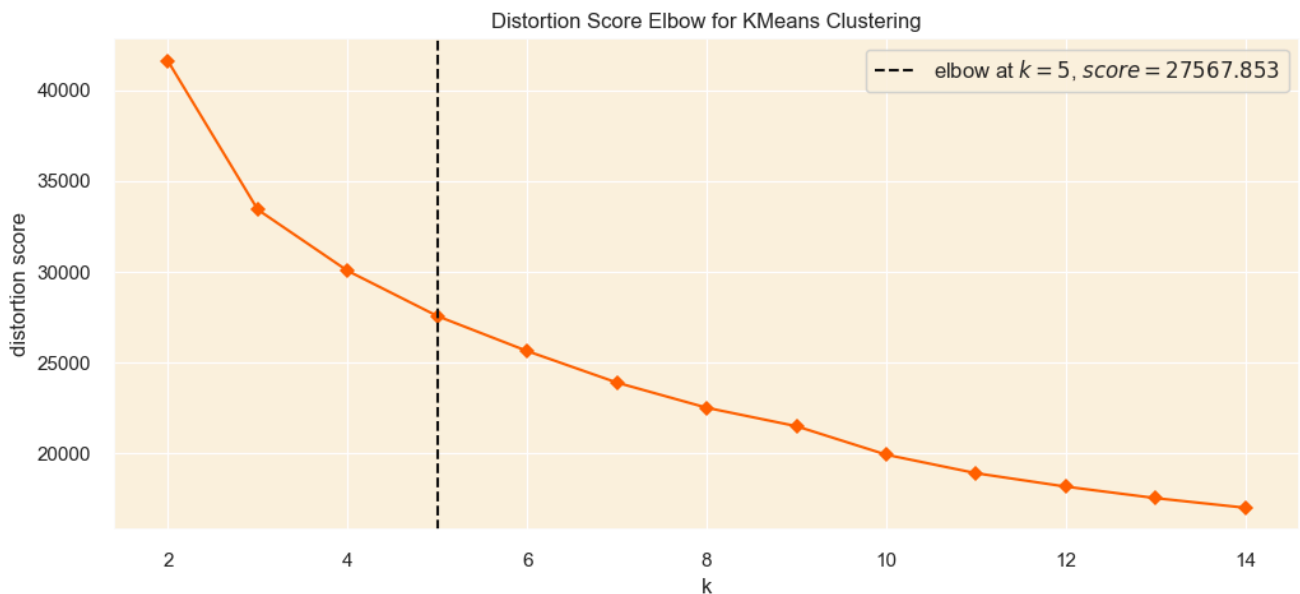
# 3. Khởi tạo mô hình phân cụm K-Means với các tham số cụ thể
# init='k-means++': chọn tâm thông minh
# n_init=10: chạy 10 lần để tìm kết quả tốt nhất
# max_iter=100: tối đa 100 vòng lặp để hội tụ
# random_state=0: cố định kết quả để tái lập được
km = KMeans(init='k-means++', n_init=10, max_iter=100, random_state=0)

# 4. Tạo khung hình (figure) và trục (axis) với kích thước mong muốn (12x5 inch)
fig, ax = plt.subplots(figsize=(12, 5))

# 5. Khởi tạo công cụ KElbowVisualizer từ thư viện YellowBrick
# k=(2, 15): Thử nghiệm số cụm từ 2 đến 14
# timings=False: Tắt biểu đồ hiển thị thời gian chạy
visualizer = KElbowVisualizer(km, k=(2, 15), timings=False, ax=ax, force_model=True)

# 6. Fit dữ liệu PCA vào bộ trục quan hóa để bắt đầu tính toán
visualizer.fit(customer_data_pca)

# 7. Hoàn thiện và hiển thị biểu đồ lên màn hình
visualizer.show();
```



Điểm Elbow được xác định bằng cách chạy K-Means với nhiều giá trị k và tính SSE (Sum of Squared Errors) cho mỗi k. Khi vẽ đồ thị SSE theo k, điểm Elbow là điểm mà mức giảm SSE bắt đầu chậm lại - tức là tăng k thêm nữa không giúp giảm lỗi đáng kể. Dựa vào đồ thị ta có thể thấy số cụm tối ưu k là 5.

Tuy nhiên, trong trường hợp này, chúng ta không thấy một điểm khuỷu tay quá rõ rệt, điều này rất phổ biến đối với dữ liệu thực tế. Từ biểu đồ, có thể thấy rằng độ quán tính (inertia) tiếp tục giảm đáng kể cho đến khi $k = 5$, chỉ ra rằng giá trị tối ưu của k có thể nằm trong khoảng từ 3 đến 7.

Để chọn được giá trị k tốt nhất trong khoảng này, chúng ta có thể sử dụng phân tích hình bóng (silhouette analysis), một phương pháp đánh giá chất lượng cụm khác. Ngoài ra, việc kết hợp các góc nhìn kinh doanh (business insights) cũng có thể giúp xác định một giá trị k mang tính thực tiễn.

Phương pháp Silhouette là gì?

Phương pháp Hình bóng (Silhouette Method) là một cách tiếp cận để tìm ra số lượng cụm tối ưu trong tập dữ liệu bằng cách đánh giá sự nhất quán bên trong các cụm và sự tách biệt của chúng so với các cụm khác. Nó tính toán hệ số Silhouette cho từng điểm dữ liệu, thước đo này cho biết mức độ tương đồng của một điểm đối với cụm của chính nó so với các cụm khác.

Hệ số Silhouette là gì?

Các bước để xác định hệ số Silhouette cho một điểm i bất kỳ:

Tính $a(i)$: Tính khoảng cách trung bình giữa điểm i và tất cả các điểm khác nằm trong cùng cụm với nó.

Tính $b(i)$: Tính khoảng cách trung bình giữa điểm i và tất cả các điểm nằm trong cụm gần nhất (không phải cụm của nó).

Tính hệ số Silhouette $s(i)$: Cho điểm i sử dụng công thức sau:

$$s(i) = \frac{b(i) - a(i)}{\max(b(i), a(i))}$$

Chú ý: Hệ số Silhouette định lượng mức độ tương đồng của một điểm đối với cụm của chính nó (độ gắn kết - cohesion) so với mức độ tách biệt của nó khỏi các cụm khác (độ tách biệt - separation). Giá trị này nằm trong khoảng từ -1 đến 1, với giá trị càng cao biểu thị rằng điểm đó nằm rất "đúng chỗ" trong cụm của mình và ít có điểm chung với các cụm lân cận.

Silhouette Method có gì vượt trội hơn so với Elbow Method?

- Phương pháp Silhouette đánh giá chất lượng cụm bằng cách xem xét cả độ gắn kết (cohesion) bên trong cụm và độ tách biệt (separation) của chúng đối với các cụm khác. Điều này cung cấp một thước đo toàn diện hơn về hiệu suất phân cụm so với Elbow Method, vốn chỉ xem xét quán tính (tổng bình phương khoảng cách bên trong cụm).
- Phương pháp Silhouette tạo ra một điểm số Silhouette định lượng trực tiếp chất lượng phân cụm, giúp việc so sánh các giá trị k khác nhau trở nên dễ dàng hơn. Ngược lại, Elbow Method dựa vào sự diễn giải chủ quan về điểm gập, điều này có thể kém tin cậy trong các trường hợp biểu đồ không hiển thị điểm khuỷu tay rõ ràng.
- Phương pháp Silhouette tạo ra một biểu diễn trực quan về các hệ số Silhouette cho từng điểm dữ liệu, cho phép nhận diện dễ dàng hơn các biến động và các điểm ngoại lai (outliers) bên trong cụm. Điều này giúp xác định số lượng cụm tối ưu với độ tin cậy cao hơn, trái ngược với Phương pháp Elbow vốn chỉ dựa vào việc quan sát bằng mắt thường trên biểu đồ quán tính.

Phương pháp thực hiện:

- Đầu tiên, chọn ra khoảng giá trị từ 3 đến 7 cho số lượng cụm (k) dựa trên kết quả của Elbow Method ở phần trước.

- Tiếp theo, vẽ biểu đồ Silhouette scores cho từng giá trị k để xác định xem giá trị nào mang lại điểm số cao nhất.
- Sau đó, để tinh chỉnh (fine-tune) việc lựa chọn giá trị k phù hợp nhất, tạo các Biểu đồ Hình bóng (Silhouette plots). Các biểu đồ này sẽ hiển thị trực quan các hệ số Silhouette cho từng điểm dữ liệu cụ thể bên trong các cụm khác nhau.

Thư viện YellowBrick sẽ được sử dụng một lần nữa để tạo ra các biểu đồ này và hỗ trợ quá trình phân tích so sánh.

```
In [52]: def silhouette_analysis(df, start_k, stop_k, figsize=(15, 16)):
    """
    Thực hiện phân tích Silhouette cho một khoảng giá trị k và trực quan hóa kết quả.
    """

    # 1. Thiết lập kích thước tổng thể cho khung hình
    plt.figure(figsize=figsize)

    # 2. Tạo Lưới bố cục (GridSpec)
    # Số hàng = (Số lượng k cần thử) + 1 hàng cho biểu đồ tổng quan
    # 2 cột để hiển thị các biểu đồ nhỏ song song
    grid = gridspec.GridSpec(stop_k - start_k + 1, 2)

    # -----
    # PHẦN 1: VẼ BIỂU ĐỒ ĐƯỜNG TỔNG QUAN (AVERAGE SCORE)
    # -----

    # Gán vị trí hàng đầu tiên (row 0), lấy cả 2 cột (:) cho biểu đồ này
    first_plot = plt.subplot(grid[0, :])

    # Thiết lập màu sắc
    sns.set_palette(['darkorange'])
    silhouette_scores = []

    # Vòng lặp chạy qua các giá trị k (ví dụ: từ 2 đến 7)
    for k in range(start_k, stop_k + 1):
        km = KMeans(n_clusters=k, init='k-means++', n_init=10, max_iter=100, random_state=0)
        km.fit(df)
        labels = km.predict(df)
        score = silhouette_score(df, labels)
        silhouette_scores.append(score)

    # Tìm ra k tốt nhất (có điểm cao nhất)
    best_k = start_k + silhouette_scores.index(max(silhouette_scores))

    # Vẽ biểu đồ đường
    plt.plot(range(start_k, stop_k + 1), silhouette_scores, marker='o')
    plt.xticks(range(start_k, stop_k + 1))
    plt.xlabel('Số lượng cụm (k)')
    plt.ylabel('Điểm Silhouette trung bình')
    plt.title('Điểm Silhouette trung bình cho các giá trị k khác nhau', fontsize=15)

    # Thêm hộp văn bản thông báo giá trị k tối ưu
    optimal_k_text = f'Giá trị k có điểm Silhouette cao nhất là: {best_k}'
    first_plot.text(
        0.95, 0.05, optimal_k_text,
        transform=first_plot.transAxes,
        fontsize=12,
        ha='right',
        bbox=dict(facecolor='#fcc36d', edgecolor='#ff6200', boxstyle='round,pad=0.5')
    )

    # -----
    # PHẦN 2: VẼ CÁC BIỂU ĐỒ HÌNH BÓNG CHI TIẾT (SUBPLOTS)
    # -----
```

```

colors = sns.color_palette("bright")

for i in range(start_k, stop_k + 1):
    # Chạy Lại K-Means cho từng k để vẽ visualizer
    km = KMeans(n_clusters=i, init='k-means++', n_init=10, max_iter=100, random_state=0)

    # Tính toán vị trí đặt biểu đồ (dòng nào, cột nào)
    # divmod giúp chia Luân phiên vào 2 cột
    row_idx, col_idx = divmod(i - start_k, 2)

    # Đặt biểu đồ vào các hàng tiếp theo (row_idx + 1)
    ax = plt.subplot(grid[row_idx + 1, col_idx])

    # Sử dụng YellowBrick để vẽ Silhouette Plot
    visualizer = SilhouetteVisualizer(km, colors=colors, ax=ax, force_model=True)
    visualizer.fit(df)

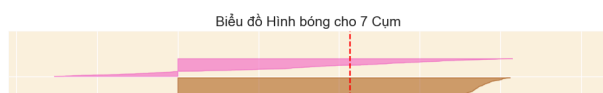
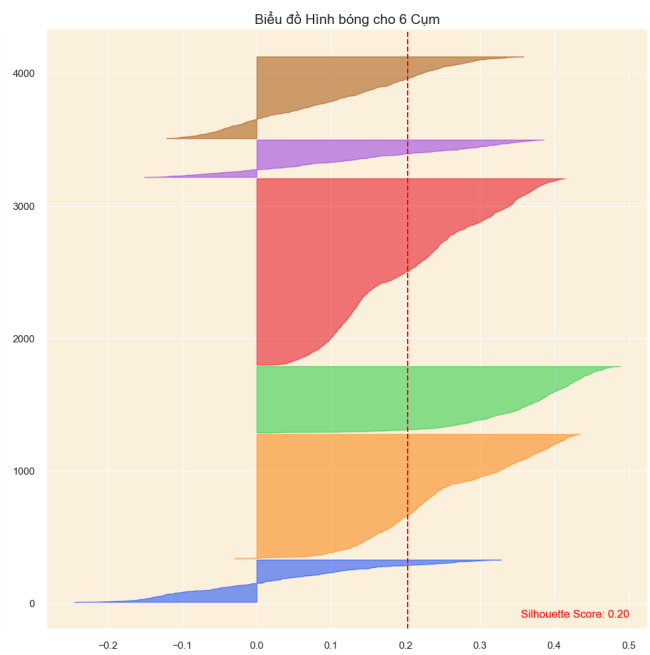
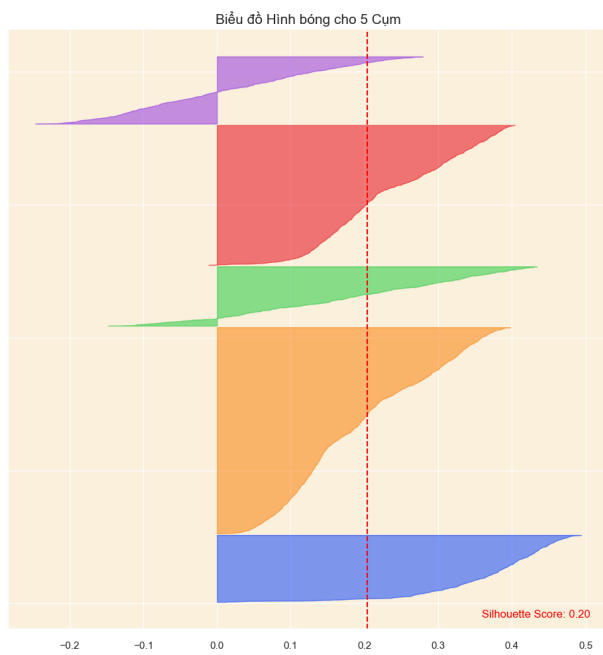
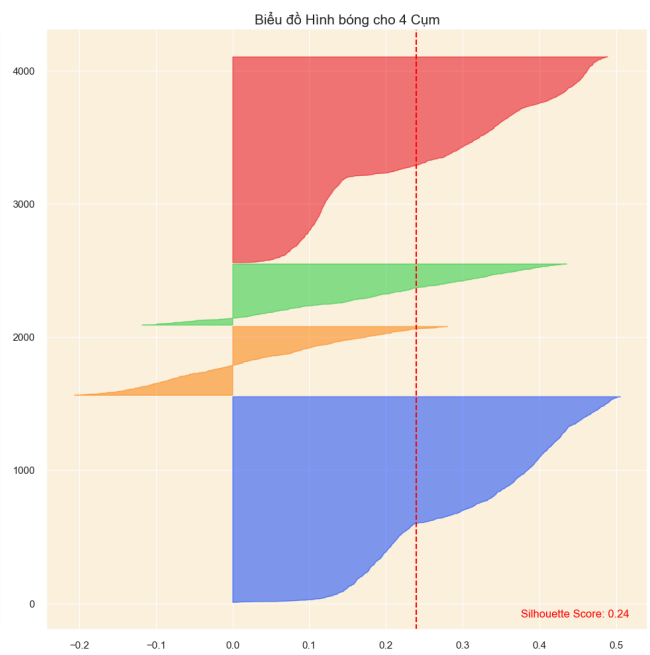
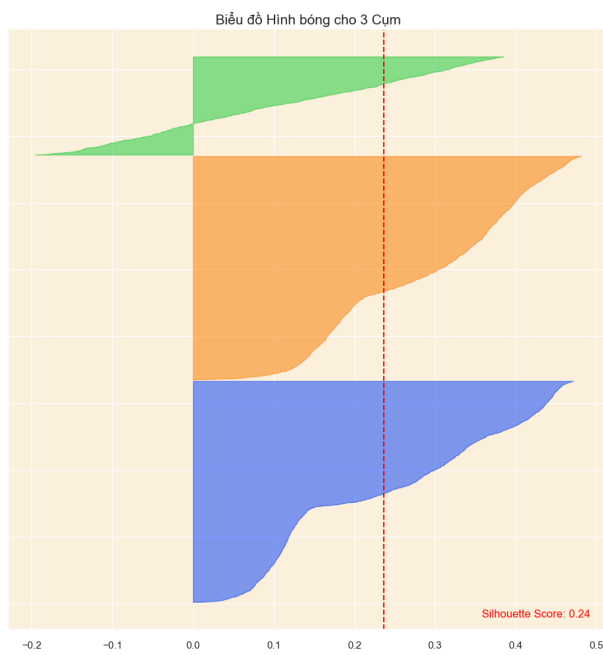
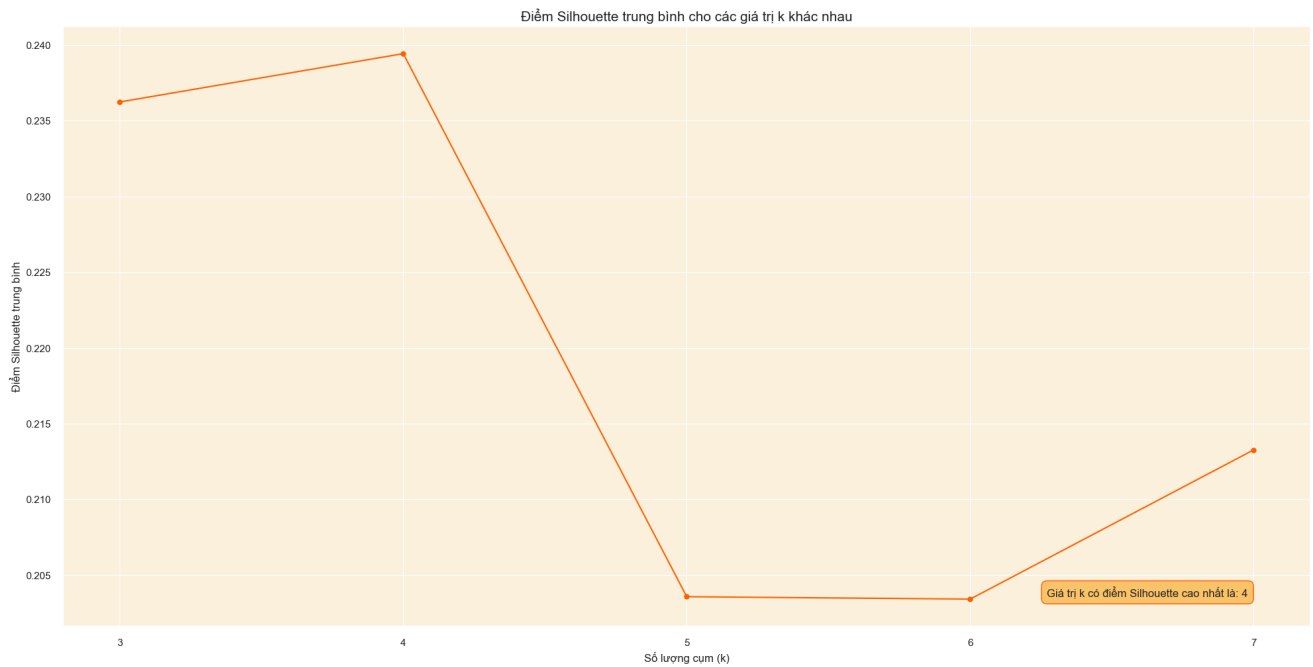
    # Ghi điểm số cụ thể lên góc biểu đồ nhỏ
    score = silhouette_score(df, km.labels_)
    ax.text(0.97, 0.02, f'Silhouette Score: {score:.2f}', fontsize=12, \
           ha='right', transform=ax.transAxes, color='red')

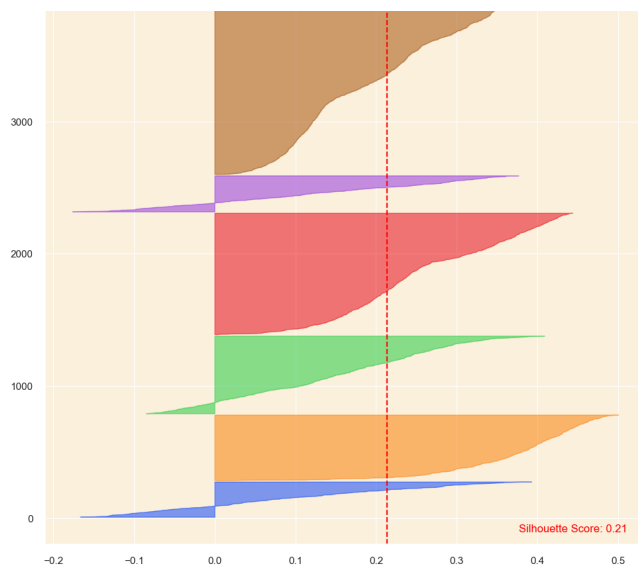
    ax.set_title(f'Biểu đồ Hình bóng cho {i} Cụm', fontsize=15)

# Căn chỉnh bố cục cho đẹp và hiển thị
plt.tight_layout()
plt.show()

```

```
In [53]: silhouette_analysis(customer_data_pca, 3, 7, figsize=(20, 50))
```





Giải thích và tiêu chí đánh giá Silhouette

Biểu đồ Silhouette được dùng để đánh giá chất lượng phân cụm và lựa chọn số lượng cụm tối ưu. Chỉ số Silhouette nằm trong khoảng $[-1, 1]$; giá trị càng gần 1 cho thấy điểm dữ liệu được phân tách rõ với các cụm khác, giá trị gần 0 phản ánh ranh giới mờ hồ giữa các cụm, còn giá trị âm biểu thị việc gán cụm sai.

Khi phân tích biểu đồ, cần chú ý các tiêu chí sau:

- Độ rộng theo chiều ngang:** Độ rộng lớn (tiến về 1) cho thấy cụm tách biệt tốt; độ rộng nhỏ hoặc âm cho thấy phân cụm không rõ ràng.
- Điểm Silhouette trung bình:** Điểm trung bình cao phản ánh phân cụm hợp lý đối với giá trị k đó.
- Sự đồng đều giữa các cụm:** Nếu các cụm có độ dày tương đối đồng đều, điều này cho thấy phân bố dữ liệu cân bằng; chênh lệch lớn về độ dày cho thấy cấu trúc cụm không ổn định.

Trong thực tế, giá trị k tối ưu thường là k cho điểm Silhouette trung bình cao nhất, đồng thời đảm bảo các cụm có cấu trúc rõ ràng và tương đối cân bằng. Phân tích này giúp lựa chọn k một cách khách quan và có cơ sở thống kê.

Vậy, dựa vào các yếu tố trên ta thấy $k=3$ là lựa chọn tối ưu nhất

9.2 Clustering Model - K-means

Trong bước này, ta sẽ áp dụng thuật toán phân cụm K-means để phân khúc khách hàng thành các cụm khác nhau dựa trên hành vi mua sắm và các đặc điểm khác của họ, sử dụng số lượng cụm tối ưu đã được xác định ở bước trước ($k=3$).

Cần lưu ý quan trọng rằng thuật toán K-means có thể gán các nhãn (labels) khác nhau cho các cụm trong mỗi lần chạy. Để giải quyết vấn đề này, thực hiện thêm một bước bổ sung là hoán đổi các nhãn dựa trên tần suất mẫu (số lượng khách hàng) trong mỗi cụm, nhằm đảm bảo việc gán nhãn được nhất quán qua các lần chạy khác nhau.

```
In [56]: # 1. Áp dụng thuật toán K-Means với số k tối ưu (k=3)
kmeans = KMeans(n_clusters = 3, init='k-means++', n_init=10, max_iter=100, random_state=0)
kmeans.fit(customer_data_pca)

# 2. Đếm số Lượng khách hàng trong mỗi cụm (Tần suất)
cluster_frequencies = Counter(kmeans.labels_)

# 3. Tạo một bảng ánh xạ (mapping) từ nhãn cũ sang nhãn mới dựa trên độ Lớn của cụm
# (Cụm đông nhất sẽ thành số 0, đông nhì thành 1...)
label_mapping = {label: new_label for new_label, (label, _) in
                  enumerate(cluster_frequencies.most_common())}
```

```
# 4. Đảo ngược hoặc tùy chỉnh lại bảng ánh xạ
# Dòng này sẽ ghi đè lên dòng trên. Đây là quy tắc thủ công để sắp xếp lại thứ tự cụm.
# {2: 1, 1: 0, 0: 2} nghĩa là: Muốn cụm đang là 1 thành 0, cụm 2 thành 1, cụm 0 thành 2.
label_mapping = {v: k for v, k in {2:1, 1:0, 0:2}.items()}

# 5. Áp dụng bảng ánh xạ để tạo ra danh sách nhãn mới
new_labels = np.array([label_mapping[label] for label in kmeans.labels_])

# 6. Gán nhãn cụm mới (cluster) vào bộ dữ liệu gốc (đã làm sạch)
customer_data_cleaned['cluster'] = new_labels

# 7. Gán nhãn cụm mới vào bộ dữ liệu PCA
customer_data_pca['cluster'] = new_labels
```

In [57]: `customer_data_cleaned.head()`

Out[57]:

	CustomerID	Days_Since_Last_Purchase	Total_Transactions	Total_Products_Purchased	Total_Spend	Average
0	12346.0	325	2	0	0.0	
1	12347.0	2	7	2458	4310.0	
2	12348.0	75	4	2332	1437.24	
3	12349.0	18	1	630	1457.55	
4	12350.0	310	1	196	294.4	

10. ĐÁNH GIÁ CỤM HÌNH THÀNH

Sau khi xác định được số lượng cụm tối ưu bằng cách sử dụng phân tích Elbow và Silhouette, ta chuyển sang bước đánh giá (evaluation) để kiểm định chất lượng của các cụm được hình thành.

Bước này rất cần thiết để xác nhận tính hiệu quả của việc phân cụm và đảm bảo rằng các cụm có tính nhất quán (coherent) và tách biệt tốt (well-separated). Ta sử dụng kỹ thuật phân tích và các chỉ số đánh giá sau:

- Trực quan hóa phân phối cụm (Số lượng khách hàng mỗi nhóm)
- Các chỉ số đánh giá:
 - Điểm Silhouette
 - Điểm Calinski Harabasz
 - Điểm Davies Bouldin

10.1 Cluster Distribution Visualization

In [58]:

```
# Tính phần trăm khách hàng trong mỗi cụm
cluster_percentage = (customer_data_pca['cluster'].value_counts(normalize=True) * 100).reset_index()
cluster_percentage.columns = ['Cluster', 'Percentage']
cluster_percentage.sort_values(by='Cluster', inplace=True)

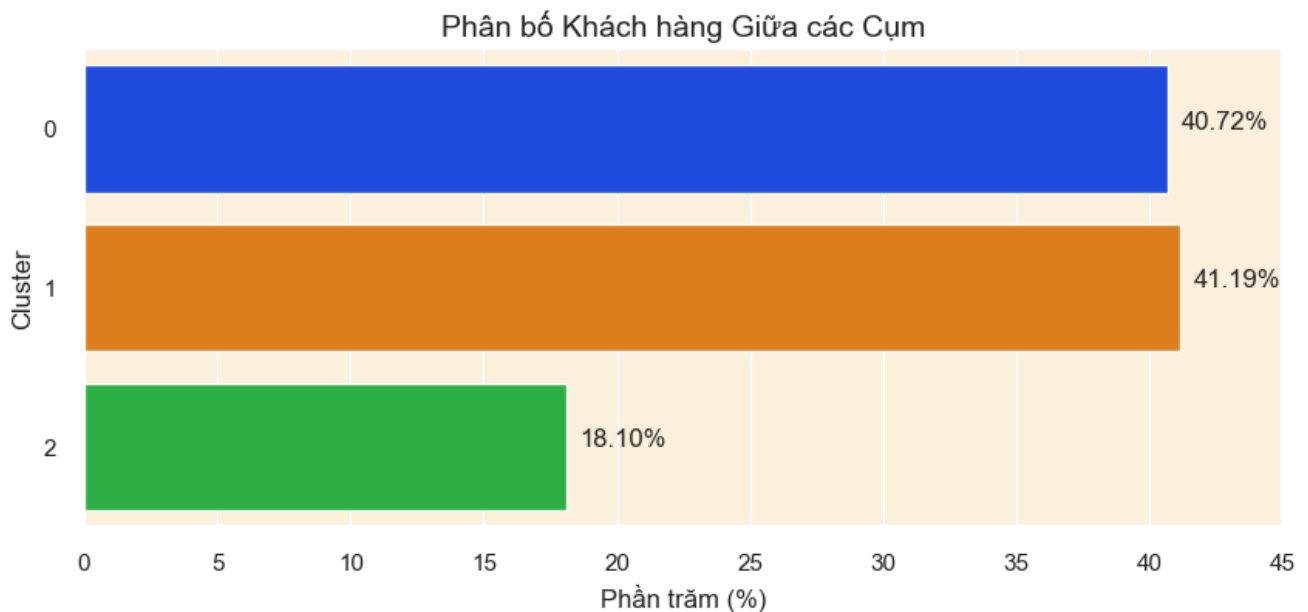
# Tạo biểu đồ thanh ngang
plt.figure(figsize=(10, 4))
colors = sns.color_palette("bright")
sns.barplot(x='Percentage', y='Cluster', data=cluster_percentage, orient='h', palette = colors)

# Thêm số phần trăm lên các thanh
for index, value in enumerate(cluster_percentage['Percentage']):
```

```
plt.text(value+0.5, index, f'{value:.2f}%')

plt.title('Phân bố Khách hàng Giữa các Cụm', fontsize=14)
plt.xticks(ticks=np.arange(0, 50, 5))
plt.xlabel('Phần trăm (%)')

# Hiển thị biểu đồ
plt.show()
```



Nhận xét

- Sự phân bố khách hàng giữa các cụm, như được thể hiện qua biểu đồ, cho thấy một cấu trúc khá cân bằng. Cụm 0 và cụm 1 mỗi nhóm chiếm khoảng 41% lượng khách hàng, trong khi cụm 2 chiếm khoảng 18%.
- Sự phân bố cân đối này chỉ ra rằng quá trình phân cụm (clustering) đã nhận diện được các mẫu hình có ý nghĩa (meaningful patterns) bên trong dữ liệu. Điều này ngụ ý rằng mỗi cụm đại diện cho một phân khúc khách hàng lớn và riêng biệt, từ đó cung cấp những thông tin chi tiết (insights) có giá trị cho các chiến lược kinh doanh trong tương lai.
- Hơn nữa, việc không có cụm nào chứa tỷ lệ phần trăm khách hàng quá nhỏ đảm bảo rằng mỗi cụm đều có ý nghĩa thống kê, chứ không chỉ đơn thuần đại diện cho các giá trị ngoại lai (outliers) hay nhiễu trong dữ liệu. Cấu trúc này cho phép chúng ta hiểu và phân tích sâu sắc hơn về các phân khúc khách hàng khác nhau, tạo điều kiện thuận lợi cho việc ra quyết định hiệu quả và có cơ sở.

10.2 Evaluation Metrics

Để đánh giá sâu hơn chất lượng của quá trình phân cụm, ta sẽ sử dụng các chỉ số đo lường sau:

- Silhouette Score:** Một thước đo dùng để đánh giá khoảng cách tách biệt giữa các cụm. Giá trị cao hơn cho thấy sự tách biệt giữa các cụm tốt hơn. Chỉ số này dao động trong khoảng từ -1 đến 1.
- Calinski-Harabasz Score:** Điểm số này được sử dụng để đánh giá độ phân tán giữa các cụm (giữa các nhóm khác nhau) và trong nội bộ cụm (sự cô đặc bên trong nhóm). Điểm số càng cao cho thấy các cụm được xác định càng rõ ràng và tách biệt.
- Davies-Bouldin Score:** Chỉ số này đánh giá độ tương đồng trung bình giữa mỗi cụm và cụm gần nó nhất. Giá trị thấp hơn cho thấy sự tách biệt cụm tốt hơn.

```
In [59]: # 1. Tính tổng số Lượng khách hàng (quan sát)
num_observations = len(customer_data_pca)
```

```
# 2. Tách các đặc trưng (features) và nhãn cụm (Labels)
# X là dữ liệu đầu vào (tọa độ các điểm), clusters là nhãn (0, 1, 2)
X = customer_data_pca.drop('cluster', axis=1)
clusters = customer_data_pca['cluster']

# 3. Tính toán các chỉ số đánh giá (Metrics)
sil_score = silhouette_score(X, clusters)
calinski_score = calinski_harabasz_score(X, clusters)
davies_score = davies_bouldin_score(X, clusters)

# 4. Tạo dữ liệu cho bảng hiển thị
table_data = [
    ["Số lượng quan sát (Customers)", num_observations],
    ["Silhouette Score", f"{sil_score:.4f}"], # Làm tròn 4 chữ số
    ["Calinski Harabasz Score", f"{calinski_score:.4f}"],
    ["Davies Bouldin Score", f"{davies_score:.4f}"]
]

# 5. In bảng kết quả
print(tabulate(table_data, headers=["Metric", "Value"], tablefmt='pretty'))
```

Metric	Value
Số lượng quan sát (Customers)	4067
Silhouette Score	0.2362
Calinski Harabasz Score	1257.1748
Davies Bouldin Score	1.3683

Nhận xét:

- Silhouette Score: Kết quả xấp xỉ 0.236, tuy chưa tiệm cận mức lý tưởng là 1, nhưng vẫn cho thấy một mức độ tách biệt tương đối (fair amount) giữa các cụm. Điều này gợi ý rằng các cụm đã có sự phân hóa nhất định, mặc dù vẫn tồn tại sự chồng lấn nhẹ (slight overlaps) giữa chúng. Nhìn chung, điểm số càng gần 1 thì các cụm càng tách biệt rõ ràng và lý tưởng hơn.
- Calinski-Harabasz: Kết quả đạt 1257.17, đây là một con số khá cao, cho thấy các cụm được xác định rất rõ ràng (well-defined). Thông thường, điểm số ở chỉ số này càng cao thì các cụm càng được định nghĩa tốt. Điều này ngụ ý rằng thuật toán phân cụm của chúng ta đã thành công trong việc tìm ra cấu trúc cốt lõi và đáng kể bên trong dữ liệu.
- Davies-Bouldin: Kết quả 1.37 là một mức điểm hợp lý, phản ánh mức độ tương đồng vừa phải giữa mỗi cụm và cụm gần nó nhất. Đối với chỉ số này, giá trị càng thấp càng tốt (thể hiện sự ít tương đồng giữa các cụm), do đó kết quả này cho thấy sự tách biệt giữa các cụm đang ở mức độ ổn định (decent separation).

Kết luận tổng quan:

Các chỉ số đánh giá cho thấy mô hình phân cụm có chất lượng tốt, với các cụm được xác định rõ nét và tách biệt tương đối.

11. PHÂN TÍCH CỤM VÀ ĐƯA RA ĐẶC ĐIỂM

Trong phần này, nội dung sẽ đi sâu vào phân tích các đặc điểm của từng cụm nhằm thấu hiểu những hành vi và sở thích riêng biệt của các phân khúc khách hàng khác nhau. Đồng thời, ta sẽ tiến hành định hình profile cho từng cụm để xác định những đặc điểm cốt lõi giúp định nghĩa nên nhóm khách hàng trong mỗi cụm đó.

Histogram Chart Approach

Chúng ta sẽ sử dụng biểu đồ phân phối (Histograms) cho từng đặc điểm được phân nhóm theo nhãn cụm, các biểu đồ này cho phép quan sát trực quan sự phân bố giá trị của từng đặc điểm trong nội bộ mỗi cụm, từ đó giúp xác nhận hoặc xây dựng lại profile khách hàng

```
In [60]: # 1. Xác định các đặc điểm (features) cần vẽ
# Lưu ý: columns[1:-1] giả định cột đầu là ID và cột cuối là Cluster.
# Để an toàn hơn, mình dùng cách loại bỏ tên cột cụ thể.
features = customer_data_cleaned.drop(columns=['CustomerID', 'cluster']).columns

# 2. Lấy danh sách các cụm và sắp xếp theo thứ tự (0, 1, 2)
clusters = sorted(customer_data_cleaned['cluster'].unique())

# 3. Thiết lập Lưới biểu đồ con (Subplots)
n_rows = len(features)
n_cols = len(clusters)

# figsize=(chiều_rộng, chiều_cao). Chiều cao tự động tăng theo số lượng đặc điểm.
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 4*n_rows))

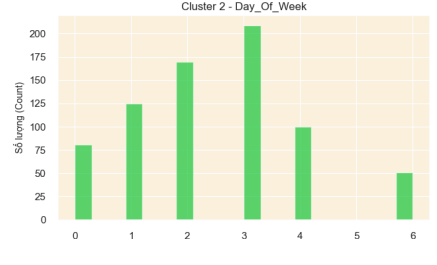
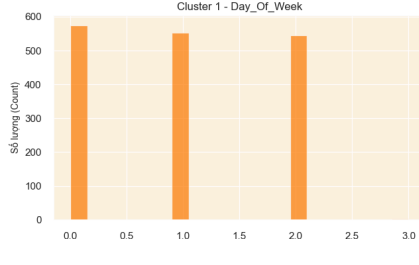
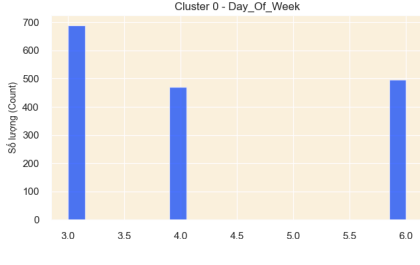
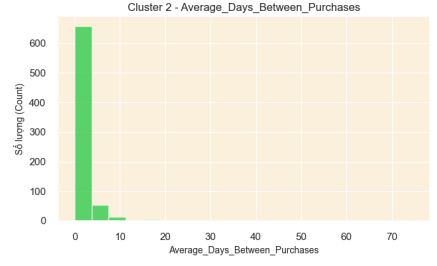
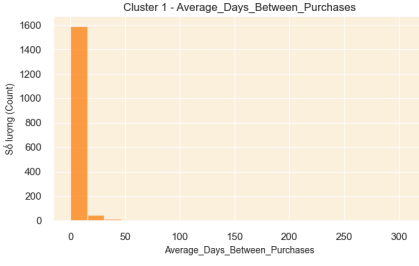
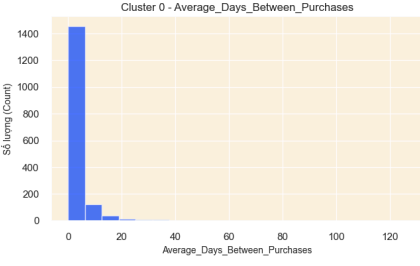
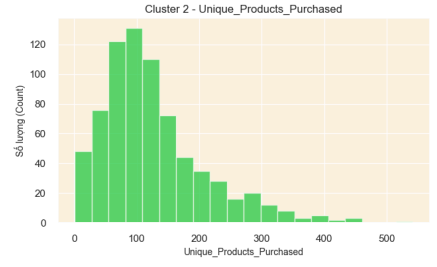
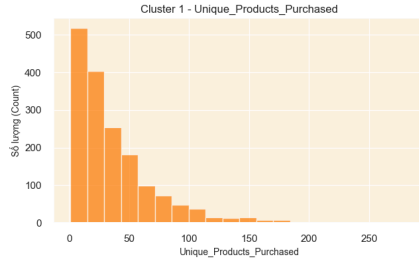
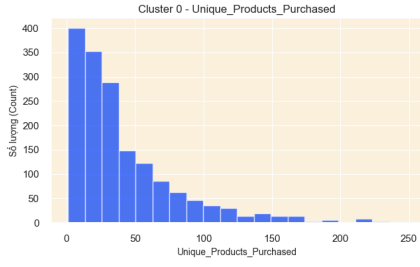
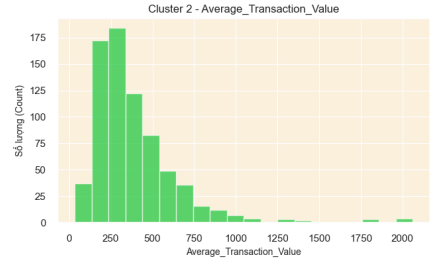
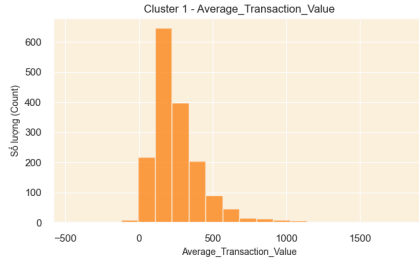
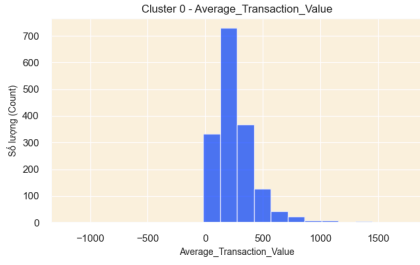
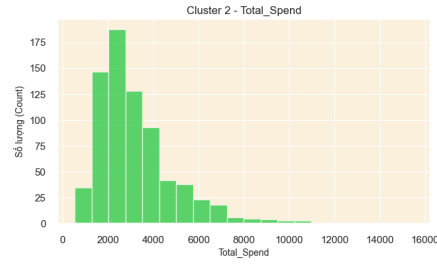
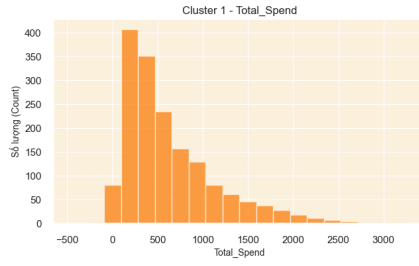
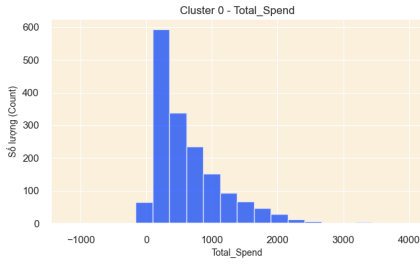
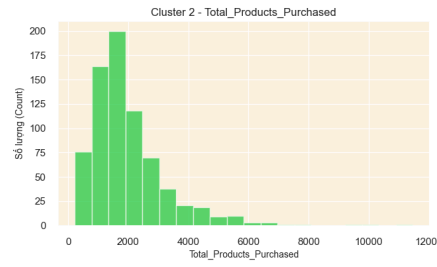
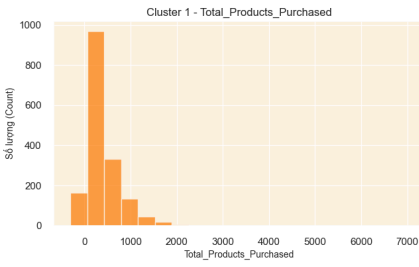
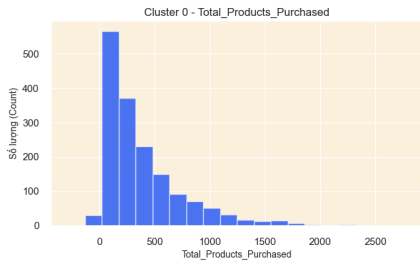
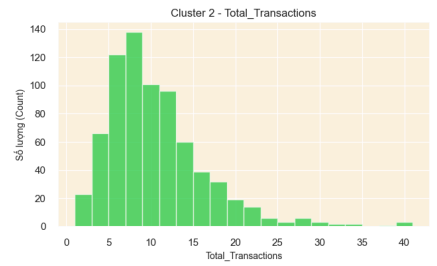
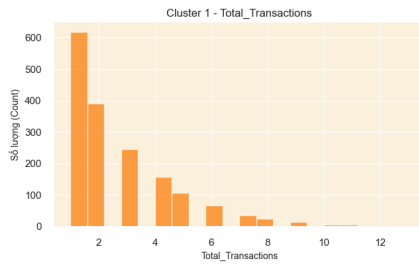
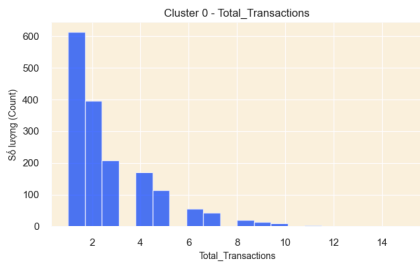
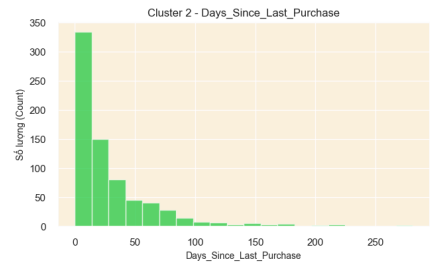
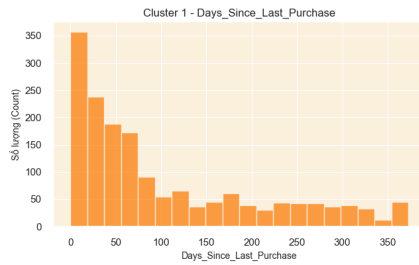
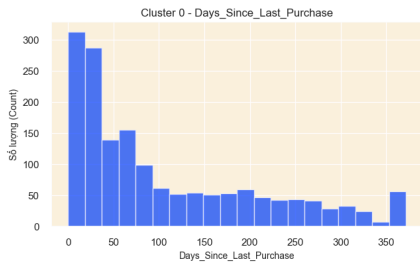
# 4. Vòng lặp vẽ biểu đồ Histogram
for i, feature in enumerate(features):
    for j, cluster in enumerate(clusters):
        # Lọc dữ liệu theo cụm và đặc điểm tương ứng
        data = customer_data_cleaned[customer_data_cleaned['cluster'] == cluster][feature]

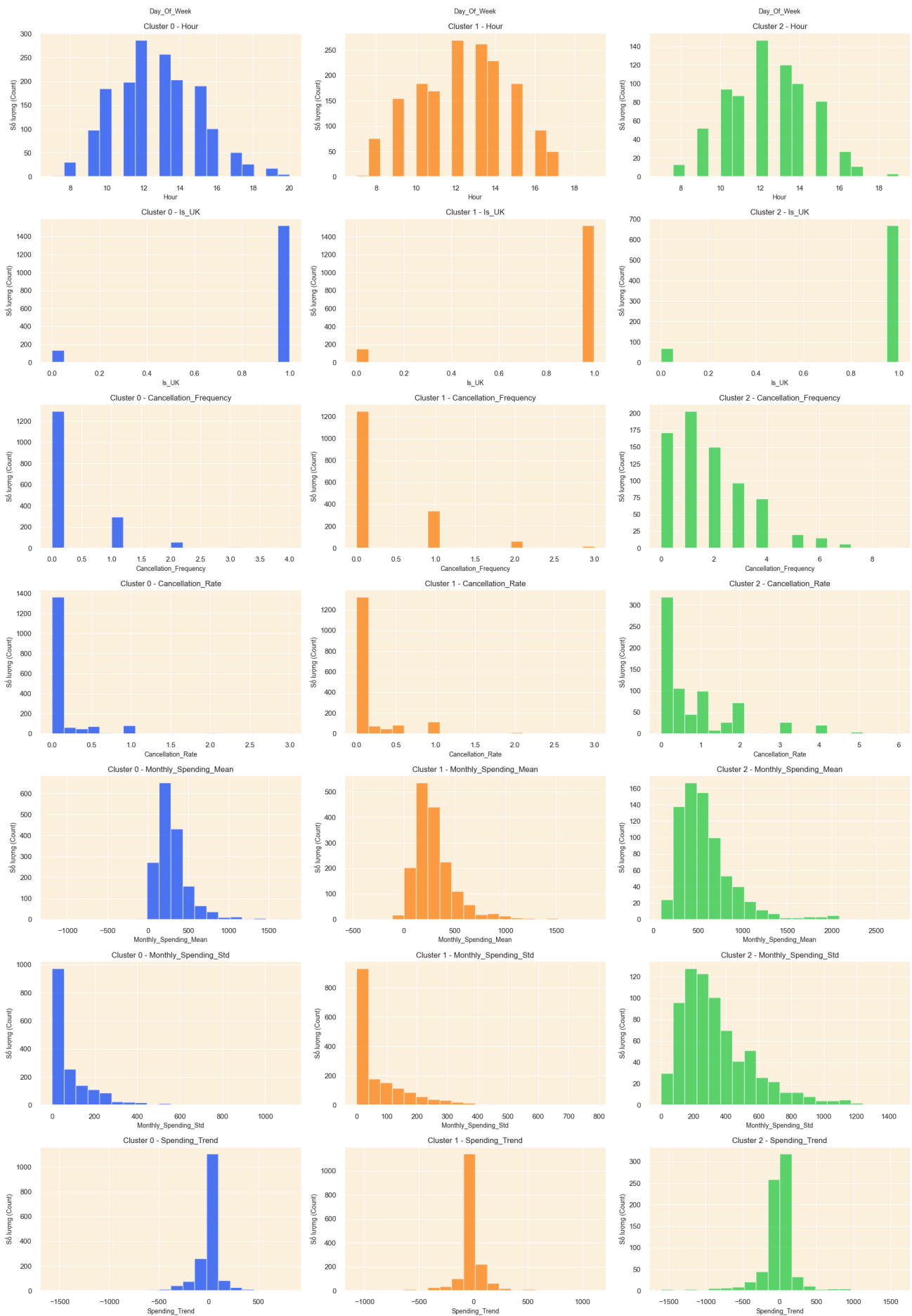
        # Vẽ histogram
        # bins=20: Chia dữ liệu thành 20 khoảng
        # color=colors[j]: Sử dụng màu tương ứng cho cụm (Đỏ, Xanh Lá, Xanh dương)
        axes[i, j].hist(data, bins=20, color=colors[j], edgecolor='w', alpha=0.7)

        # Thiết lập tiêu đề và nhãn
        axes[i, j].set_title(f'Cluster {cluster} - {feature}', fontsize=12)
        axes[i, j].set_xlabel(feature, fontsize=10)
        axes[i, j].set_ylabel('Số lượng (Count)', fontsize=10)

# 5. Điều chỉnh bố cục để tránh các chữ bị chồng lên nhau
plt.tight_layout()

# 6. Hiển thị biểu đồ
plt.show()
```





Dựa trên đồ thị chi tiết ở trên ta có thể xác định lại đặc điểm của từng cụm khách hàng và xây dựng như sau:

Cluster 0: Casual Weekend Shoppers

(Khách hàng Mua sắm Cuối tuần Ngẫu hứng - Nhóm Ổn định)

- **Hành vi mua sắm:** Nhóm này mua sắm ít thường xuyên nhất và chi tiêu ít hơn so với các nhóm khác. Họ thường mua số lượng sản phẩm ít trong mỗi giao dịch.
- **Thời gian ưa thích:** Có xu hướng rõ rệt trong việc mua sắm vào cuối tuần (thứ 7, Chủ nhật), có thể kết hợp với việc đi dạo phố hoặc xem hàng (window shopping).
- **Độ ổn định:** Thói quen chi tiêu rất ổn định theo thời gian, ít có biến động lớn giữa các tháng.
- **Tâm lý:** Họ hiếm khi hủy đơn hàng, cho thấy hành vi mua sắm rất dứt khoát (đã chọn là mua).
- **Giá trị:** Khi họ mua, giá trị đơn hàng thường thấp (Low basket value).

Cluster 1: Occasional Big Spenders

(Khách hàng Chi tiêu Lớn Không thường xuyên - Nhóm Tiềm năng Tăng trưởng)

- **Hành vi mua sắm:** Không mua sắm thường xuyên, nhưng "đã mua là mua lớn". Họ chi một khoản đáng kể cho mỗi lần giao dịch và mua đa dạng các loại sản phẩm.
- **Xu hướng:** Biểu đồ cho thấy xu hướng chi tiêu đang **tăng lên** (Rising trend), thể hiện sự quan tâm ngày càng lớn hoặc sự đầu tư nhiều hơn vào các sản phẩm của bạn.
- **Thời gian & Địa điểm:** Thích mua sắm vào cuối ngày (sau giờ làm việc) và chủ yếu đến từ Vương quốc Anh.
- **Tâm lý:** Tỷ lệ hủy đơn hàng ở mức trung bình. Điều này có thể hiểu được vì họ chi tiêu lớn nên thường cân nhắc kỹ lưỡng hơn hoặc xem xét lại quyết định mua hàng.
- **Sản phẩm:** Các đơn hàng thường có giá trị lớn, cho thấy họ ưa chuộng các sản phẩm chất lượng cao hoặc cao cấp.

Cluster 2: Eager Early-Bird Shoppers

(Khách hàng Mua sắm Sớm Nhiệt huyết - Nhóm VIP Có Rủi ro)

- **Hành vi mua sắm:** Đây là nhóm chi tiêu mạnh nhất (High spenders). Họ mua rất nhiều loại sản phẩm độc đáo và thực hiện vô số giao dịch.
- **Tâm lý:** Mặc dù chi nhiều, họ lại có tỷ lệ hủy đơn hàng rất cao. Điều này cho thấy hành vi mua sắm bốc đồng (impulsive buying) – đặt nhanh nhưng hủy cũng nhanh.
- **Thời gian ưa thích:** Thường mua sắm vào sáng sớm, có thể là tranh thủ trước giờ làm việc hoặc để săn các ưu đãi sớm (early bird deals).
- **Độ biến động:** Mẫu hình chi tiêu rất khó đoán, biến động mạnh giữa các tháng.
- **Cảnh báo:** Đáng chú ý, xu hướng chi tiêu của họ đang có dấu hiệu **giảm nhẹ**. Đây là tín hiệu cảnh báo rằng nhóm khách hàng giá trị cao này có thể đang thay đổi thói quen hoặc rời bỏ thương hiệu.

12. HỆ THỐNG ĐỀ XUẤT SẢN PHẨM

Ở giai đoạn cuối, dự án hướng đến xây dựng hệ thống gợi ý sản phẩm nhằm cải thiện trải nghiệm mua sắm trực tuyến. Hệ thống hoạt động dựa trên hành vi mua hàng đặc trưng theo từng cụm khách hàng (clusters).

Trước đó, khoảng 5% khách hàng ngoại lai (outliers) đã được tách riêng vào tập outliers_data. Do đó, hệ thống sẽ tập trung phân tích nhóm 95% khách hàng cốt lõi, xác định top sản phẩm bán chạy trong từng cụm, và đề xuất top 3 sản phẩm phổ biến nhất mà khách hàng chưa từng mua.

Phương pháp này hỗ trợ:

- Cá nhân hóa sản phẩm đề xuất
- Tiếp thị mục tiêu (targeted marketing)
- Tăng trải nghiệm và doanh số

Đối với nhóm khách hàng ngoại lai, chiến lược khởi tạo sẽ là đề xuất sản phẩm ngẫu nhiên để kích hoạt tương tác ban đầu.

```
In [61]: # Bước 1: Trích xuất CustomerID của các ngoại lai (outliers) và Loại bỏ giao dịch của họ khỏi bảng c
outlier_customer_ids = outliers_data['CustomerID'].astype('float').unique()
df_filtered = df[~df['CustomerID'].isin(outlier_customer_ids)]

# Bước 2: Đảm bảo kiểu dữ liệu CustomerID đồng nhất giữa hai bảng trước khi gộp (merge)
customer_data_cleaned['CustomerID'] = customer_data_cleaned['CustomerID'].astype('float')

# Bước 3: Gộp dữ liệu giao dịch với dữ liệu khách hàng để Lấy thông tin cụm (cluster) cho mỗi giao d
merged_data = df_filtered.merge(customer_data_cleaned[['CustomerID', 'cluster']], on='CustomerID', h

# Bước 4: Xác định Top 10 sản phẩm bán chạy nhất trong mỗi cụm dựa trên tổng số Lượng bán ra
best_selling_products = merged_data.groupby(['cluster', 'StockCode', 'Description'])['Quantity'].sum
best_selling_products = best_selling_products.sort_values(by=['cluster', 'Quantity'], ascending=[Tru
top_products_per_cluster = best_selling_products.groupby('cluster').head(10)

# Bước 5: Tạo hồ sơ các sản phẩm đã được mua bởi từng khách hàng trong mỗi cụm
customer_purchases = merged_data.groupby(['CustomerID', 'cluster', 'StockCode'])['Quantity'].sum().r

# Bước 6: Tạo gợi ý cho từng khách hàng trong mỗi cụm
recommendations = []

for cluster in top_products_per_cluster['cluster'].unique():
    # Lấy danh sách Top sản phẩm của cụm hiện tại
    top_products = top_products_per_cluster[top_products_per_cluster['cluster'] == cluster]

    # Lấy danh sách khách hàng thuộc cụm hiện tại
    customers_in_cluster = customer_data_cleaned[customer_data_cleaned['cluster'] == cluster]['Custo

    for customer in customers_in_cluster:
        # Xác định các sản phẩm khách hàng NÀY đã mua
        customer_purchased_products = customer_purchases[
            (customer_purchases['CustomerID'] == customer) &
            (customer_purchases['cluster'] == cluster)
        ][['StockCode']].tolist()

        # Tìm các sản phẩm trong Top bán chạy mà khách hàng CHƯA mua
        top_products_not_purchased = top_products[~top_products['StockCode'].isin(customer_purchased
        top_3_products_not_purchased = top_products_not_purchased.head(3)

        recommendations.append([customer, cluster] +
                                top_3_products_not_purchased[['StockCode', 'Description']].values.flat

# Bước 7: Tạo DataFrame từ danh sách gợi ý và gộp lại với dữ liệu khách hàng gốc
recommendations_df = pd.DataFrame(recommendations, columns=[
    'CustomerID', 'cluster',
    'Rec1_StockCode', 'Rec1_Description',
    'Rec2_StockCode', 'Rec2_Description',
    'Rec3_StockCode', 'Rec3_Description'
])

customer_data_with_recommendations = customer_data_cleaned.merge(recommendations_df, on=['CustomerID
```

```
In [62]: # Hiển thị 10 dòng ngẫu nhiên để kiểm tra kết quả
# Chỉ hiển thị 6 cột cuối (các cột gợi ý) để dễ nhìn
print("Bảng mẫu Gợi ý sản phẩm cho khách hàng:")
display(customer_data_with_recommendations.set_index('CustomerID').iloc[:, -6:].sample(10, random_st
```

Bảng mẫu Gợi ý sản phẩm cho khách hàng:

	Rec1_StockCode	Rec1_Description	Rec2_StockCode	Rec2_Description	Rec3_StockCode	Rec3_Descr
CustomerID						
15746.0	84879	ASSORTED COLOUR BIRD ORNAMENT	15036	ASSORTED COLOURS SILK FAN	85123A	WHITE HAI HEART T- HI
15728.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	15036	ASSI COLOUR
17462.0	18007	ESSENTIAL BALM 3.5G TIN IN ENVELOPE	84879	ASSORTED COLOUR BIRD ORNAMENT	17003	BROCAD
17420.0	18007	ESSENTIAL BALM 3.5G TIN IN ENVELOPE	84879	ASSORTED COLOUR BIRD ORNAMENT	17003	BROCAD
15342.0	18007	ESSENTIAL BALM 3.5G TIN IN ENVELOPE	84879	ASSORTED COLOUR BIRD ORNAMENT	17003	BROCAD
14335.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	15036	ASSI COLOUR
15370.0	22616	PACK OF 12 LONDON TISSUES	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	85099B	JUMBO BA RETR
17604.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	15036	ASSI COLOUR
17828.0	22616	PACK OF 12 LONDON TISSUES	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	85099B	JUMBO BA RETR
13229.0	84077	WORLD WAR 2 GLIDERS ASSTD DESIGNS	84879	ASSORTED COLOUR BIRD ORNAMENT	15036	ASSI COLOUR

13. KẾT LUẬN

Đồ án đã xây dựng được **Hệ thống Gợi ý Thương mại Điện tử (Ecommerce Recommendation Engine)** dựa trên hành vi tiêu dùng của khách hàng và đạt được các kết quả chính sau:

1. Xử lý dữ liệu toàn diện:

- Làm sạch và chuẩn hóa bộ dữ liệu giao dịch lớn (hơn 540.000 dòng), xử lý hiệu quả các vấn đề về dữ liệu bị thiếu (`CustomerID`), dữ liệu trùng lặp và các giao dịch bị hủy để đảm bảo độ chính xác cho mô hình.
- Phát hiện và xử lý các bất thường trong mã sản phẩm (`StockCode`), tạo tiền đề vững cho việc phân tích.

2. Phân khúc khách hàng hiệu quả:

- Áp dụng phương pháp phân tích **RFM** (Recency, Frequency, Monetary) kết hợp với thuật toán máy học **K-Means Clustering** để phân chia khách hàng thành các nhóm riêng biệt dựa trên thói quen mua sắm.
- Sử dụng các kỹ thuật như **PCA** (giảm chiều dữ liệu) và **Elbow Method/Silhouette Score** để tối ưu hóa số lượng cụm và trực quan hóa kết quả phân nhóm rõ ràng.

3. Hệ thống gợi ý thực tiễn:

- Từ kết quả phân khúc, hệ thống có khả năng đưa ra các gợi ý sản phẩm được cá nhân hóa cho từng nhóm khách hàng (thể hiện qua các kết quả `Rec_StockCode` và `Rec_Description` ở cuối bài).
- Hệ thống giúp doanh nghiệp thấu hiểu khách hàng, từ đó tối ưu hóa chiến lược marketing và gia tăng doanh thu bán hàng.

14. TÀI LIỆU THAM KHẢO

[1] Các file bài giảng môn Python cho Khoa học dữ liệu của ThS. Hà Văn Thảo.

[2] Project tham khảo tại:

<https://www.kaggle.com/code/farzadnekouei/customer-segmentation-recommendation-system/notebook>

[3] Website tham khảo các thư viện được sử dụng: <https://scikit-learn.org/stable/>

[4] Blog: <https://machinelearningcoban.com/> - Vũ Khắc Tiệp